

Appunti del corso di **Linguaggi**

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis **Matteo Drago**

Indice

1	Introduzione	1
1.1	Perchè studiare i PL?	1
1.2	Cosa sono i PL?	2
1.3	Cosa significa programmare?	2
1.4	Contesti di sviluppo	3
1.5	Classificazione dei PL	3
1.6	Implementazione di un PL	4
1.7	Realizzare una macchina astratta	5
1.7.1	Soluzione interpretativa	7
1.7.1.1	Operazioni	7
1.7.1.2	Struttura	8
1.7.2	Soluzione compilativa	8
1.7.2.1	Struttura	9
1.7.3	Soluzione ibrida	9
2	Descrivere i linguaggi di programmazione	11
2.1	Sintassi	11
2.1.1	Descrivere la sintassi	12
2.1.2	Grammatiche CF	12
2.1.2.1	Notazione BNF	14
2.1.3	Descrivere un semplice linguaggio imperativo	14
2.1.3.1	Analisi semantica	15
2.1.4	Categorie sintattiche	15
2.1.4.1	Espressioni	16
2.1.4.2	Comandi	16
2.1.4.3	Dichiarazioni	16
2.1.5	Descrivere un semplice linguaggio implementato	17
2.1.5.1	Sintassi	17
2.2	Semantica	18
2.2.1	Induzione matematica	18
2.2.2	Induzione strutturale	19
2.2.3	Proprietà della semantica	19
2.2.4	Tipi di semantica	19
2.2.4.1	Semantica denotazionale	20
2.2.4.2	Semantica assiomatica	21
2.2.4.3	Semantica operativa	22

3	Espressioni	24
3.1	Dscrivere le espressioni	24
3.1.1	Notazione	24
3.1.1.1	Notazione in-fissa	25
3.1.1.2	Notazione pre-fissa	25
3.1.1.3	Notazione post-fissa	26
3.1.2	Regole di precedenza	26
3.1.3	Regole di associatività	26
3.1.4	Valutazione delle espressioni	27
3.2	Sintassi	27
3.3	Semantica	28
3.3.1	Regole di transizione: espressioni aritmetiche	28
3.3.1.1	Regole di transizione: espressioni booleane	29
4	Identificatori	31
4.1	Legame	32
4.1.1	Tipi di binding	33
4.1.2	Tempo di binding	33
4.2	Ambiente	33
4.3	Sintassi	33
4.3.1	Identificatore libero	34
4.4	Semantica	35
4.4.1	Regole di transizione	35
4.4.2	Tipo di dati	36
4.4.2.1	Legami di tipo	37
4.4.3	Semantica statica	37
4.4.3.1	Ambiente statico	37
4.4.3.2	Semantica statica delle espressioni	37
4.4.3.3	Compatibilità di ambienti	39
5	Dichiarazioni	40
5.1	Sintassi	40
5.1.1	Modifica dell'ambiente	42
5.1.2	Identificatori definiti	42
5.1.3	Identificatori liberi	43
5.2	Semantica	43
5.2.1	Semantica statica	43
5.2.2	Semantica dinamica	44
6	Variabili	47
7	Comandi	49
7.1	Sintassi	49
7.2	Semantica	49
7.2.1	Semantica statica	49
7.2.2	Memoria	50
7.2.3	Aggiornamento della memoria	51
7.2.4	Semantica dinamica	51
7.3	Strutture di controllo	53

7.3.1	Comandi di selezione	53
7.3.1.1	Selettori a due vie	53
7.3.1.1.1	Semantica statica	54
7.3.1.1.2	Semantica dinamica	54
7.3.1.2	Selettori a più vie	54
8	Procedure	55
8.1	Regole di scope	56
8.1.1	Scope statico	57
8.1.2	Scope dinamico	58
8.2	Visibilità delle dichiarazioni	58

Introduzione

Il problema principale dell'informatica consiste nel voler far eseguire ad una macchina algoritmi che manipolano dati.

macchina $\xrightarrow{\text{esegue}}$ algoritmo $\xrightarrow{\text{manipolazione}}$ dati

Il primo meccanismo che viene associato alla storia dei computer è la **macchina di Babbage**, che può essere visto come il primo "computer" perchè nasce per eseguire calcoli matematici complessi in modo automatico.

Sucessivamente, c'è un'evoluzione e il computer viene riconosciuto come macchina programmabile (dare delle istruzioni alla macchina per ottenere un determinato risultato).

In questo contesto nasce **Enigma**, uno strumento per decodificare un codice mediante un algoritmo di forza bruta che tenta tutte le combinazioni.

La prima architettura che darà origine all'architettura dei computer moderni è l'**architettura di Von Neumann**, un'architettura hardware che comprende solo il linguaggio binario, un linguaggio costituito esclusivamente da stringhe di bit (0 e 1) e che può essere utilizzato per codificare qualunque altro linguaggio, ma non è comprensibile dall'essere umano.

Questo fa sì che l'uomo per poter interagire e programmare macchine, necessita di un'evoluzione dei linguaggi, ha bisogno di costruire un linguaggio comprensibile in modo più immediato.

1.1 Perchè studiare i PL?

Le motivazioni sono diverse:

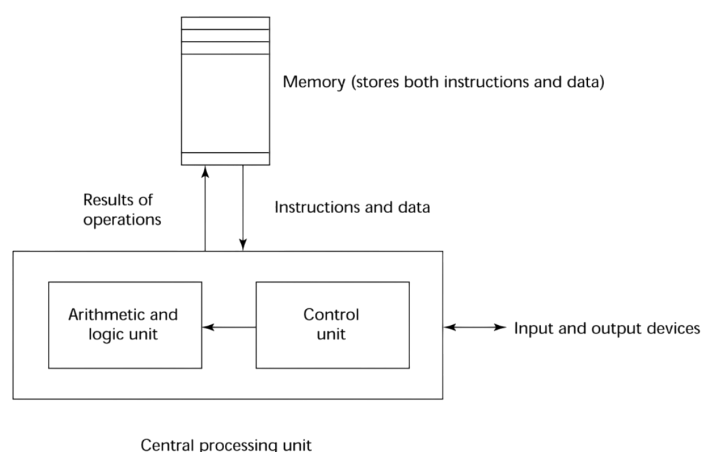
- ▷ migliorare la capacità di esprimere idee in forma algoritmica
- ▷ scegliere appropriatamente un linguaggio
- ▷ migliorare la capacità di studiare o progettare nuovi linguaggi o costrutti
- ▷ comprendere meglio il significato dell'implementazione dei linguaggi di programmazione

1.2 Cosa sono i PL?

La nascita e l'evoluzione dei linguaggi di programmazione è, anche, dovuto, all'architettura della macchina che bisogna programmare.

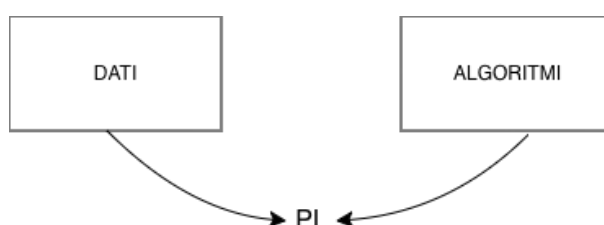
I computer moderni nascono dall'architettura di Von Neumann.

Questa è una tipologia di architettura hardware con programma memorizzato, dove i dati e istruzioni condividono la stessa area di memoria.



1.3 Cosa significa programmare?

Il linguaggio di programmazione deve essere uno strumento che permetta di descrivere algoritmi e rappresentare i dati.



Il linguaggio binario è un linguaggio di programmazione in quanto permette di rappresentare algoritmi e dati, ma non è umanamente utilizzabile o interpretabile, in quanto potrebbe essere usata come dato o un dato interpretato come istruzione.

Ciò significa, che un linguaggio di programmazione deve permettere di superare queste difficoltà, fornendo una chiara distinzione tra dato e istruzione.

Definizione: algoritmo

Un **algoritmo** è una sequenza infinita di passi di computazione, ovvero scompone un calcolo complesso in passi elementari.

Un algoritmo è un concetto astratto che trova forma concreta in un programma (sequenza finita di istruzioni).

Definizione: dati

I **dati** sono informazioni memorizzate concretamente in celle di memoria e astratte in elementi che il linguaggio può manipolare (variabili).

Definizione: programma

Un **programma** è la concretizzazione di un algoritmo che manipola dati.

Definizione: linguaggio di programmazione

Un **linguaggio di programmazione** è una notazione per scrivere un programma, ovvero serve a dare forma concreta agli algoritmi.

Il programma, quindi, costituisce la **sintassi**.

Un programma è necessariamente un oggetto finito (sempre concreto), mentre il calcolo di una funzione richiede l'esecuzione di un insieme di passi primitivi, che, però possono essere infiniti.

Ciò significa che un programma è una rappresentazione finita di un insieme, potenzialmente infinito, di passi primitivi di computazione.

Nota

I linguaggi di programmazione devono dare specifica finita di oggetti potenzialmente infiniti.

1.4 Contesti di sviluppo

Il contesto in cui il linguaggio deve operare determina le funzionalità che deve gestire efficacemente:

- ▷ **applicazioni scientifiche** (es. Fortran)
- ▷ **applicazioni economiche** (es. COBOL)
- ▷ **applicazioni artificiali** (es. LISP)
- ▷ **programmazione di sistemi** (es. C)
- ▷ **web software** (es. HTML (markup), PHP (scripting), ecc.)

1.5 Classificazione dei PL

I linguaggi di programmazione si possono classificare in:

- ◊ **basso livello**: linguaggi con caratteristiche specifiche dipendenti dall'architettura che si sta programmando (es. linguaggio binario, Assembly)
- ◊ **alto livello**: linguaggi che permettono una programmazione strutturata in cui dati e istruzioni hanno rappresentazioni diverse

I linguaggi ad alto livello si possono classificare in base a:

- **paradigma di programmazione**

- linguaggi procedurali: codice diviso in subrutine che eseguono compiti semplici basati su salti condizionali (goto) [es. COBOL]
- linguaggi strutturati (imperativi): procedurale senza goto (es. C, Pascal, ADA, ec..)
- linguaggi orientati agli oggetti: classe elemento centrale della programmazione. Un linguaggio deve avere tre proprietà:
 - * incapsulamento
 - * eriditarietà
 - * polimorfismo
- linguaggi dichiarativi: basati su componenti della matematica
 - * logici: basati sulla logica proposizionale del primo ordine, in cui si effettuano calcoli tramite dimostrazione di dati (es. PROLOG)
 - * matematici: basati sul concetto di applicazione di funzione (es. LSIP, ML)

- **termini generazionali**

- I° generazione:



- II° generazione:

1.6 Implementazione di un PL

Implementare un linguaggio significa eseguire un programma nel linguaggio di programmazione in una macchina.

Sicuramente, il modo in cui deve essere implementato è direttamente collegato all'architettura che deve programmare, ma anche al paradigma (metodologie) che il linguaggio di programmazione utilizza per costruire programmi.

Implementare un linguaggio significa realizzare la macchina (astratta) che interpreta il linguaggio.

Nota

La macchina è astratta perchè, per rappresentare la macchina fisica (concreta), utilizza un linguaggio che è più astratto del linguaggio binario.

Definizione: macchina astratta

Dato un linguaggio L di programmazione, la **macchina astratta** M_L per L , è un insieme di strutture dati e algoritmi (è un programma) che permettono di memorizzare ed eseguire i programmi scritti in L .

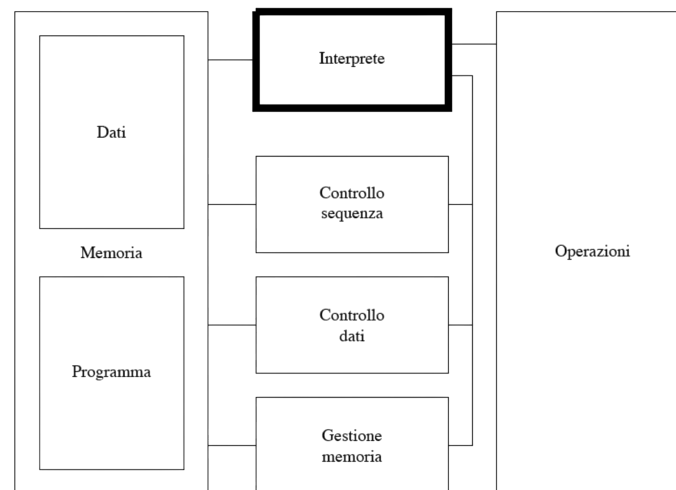


Figura 1.1: struttura della macchina astratta

Il linguaggio L , riconosciuto (interpretato) dalla macchina astratta M_L , viene anche chiamato **linguaggio macchina**.

Definizione: linguaggio macchina

Il **linguaggio macchina** è l'insieme di tutte le stringhe interpretabili da M .

1.7 Realizzare una macchina astratta

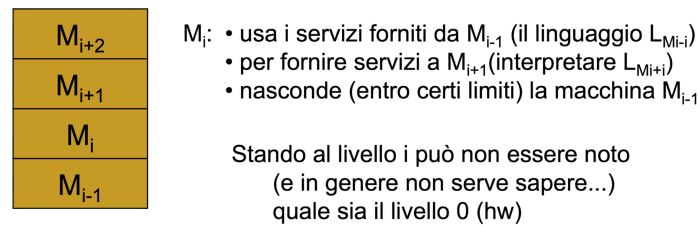
Una qualsiasi macchina astratta per L , per essere eseguita, deve utilizzare qualche dispositivo fisico che concretamente esegue le istruzioni di L .

Questo significa che in qualunque sistema informatico esiste sempre una macchina HW, ma non significa che tutte le macchine sono realizzate a livello HW, anzi per poter controllare la complessità di un sistema informatico, la realizzazione della macchina astratta può avvenire a vari livelli:

- ▷ **HW:** macchina realizzata in forma embedded e il linguaggio macchina è il linguaggio fisico/binario
 - massima velocità
 - nessuna flessibilità
- ▷ **FW:** macchina realizzata mediante simulazione/emulazione, ovvero strutture dati e algoritmi sono microprogrammi
 - veloce
 - più flessibile della realizzazione HW
- ▷ **SW:** macchina realizzata mediante simulazione, ovvero strutture dati e algoritmi sono a loro volta dei programmi
 - minore velocità
 - maggiore flessibilità

Una macchina (sistema) complessa è suddivisa in livelli di astrazione cooperanti, ma sequenziali e indipendenti.

Ciascun livello è definito da un linguaggio L che è l'insieme delle istruzioni che il livello mette a disposizione per i livelli successivi/superiori.



Tutte le istruzioni ad un livello i sono implementate da programmi a livelli $j < i$.

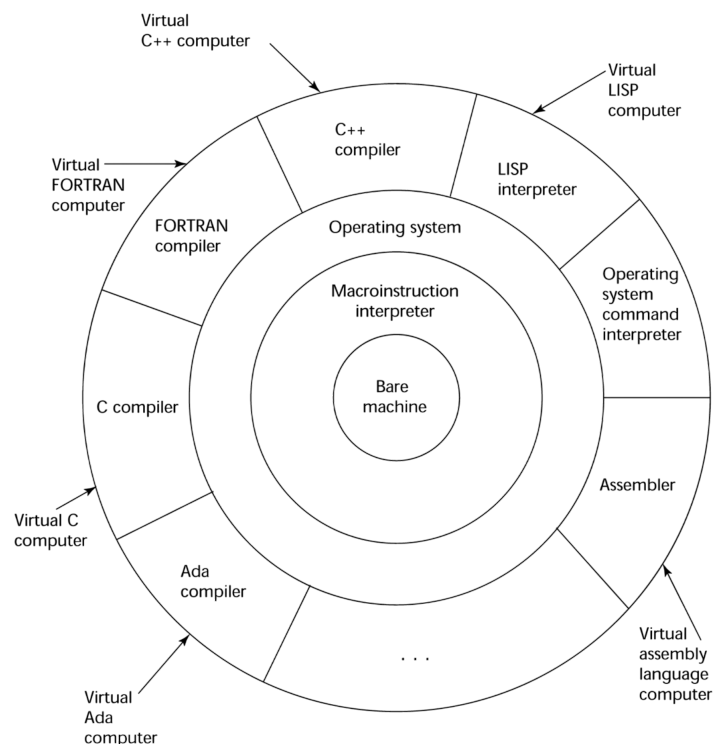


Figura 1.2: livelli di astrazione dei PL

Per Implementare un linguaggio L , dobbiamo realizzare una macchina M_L in grado di eseguirlo. Tali macchine differiscono nel mondo in cui la macchina astratta realizzata e nelle strutture dati utilizzate.

La realizzazione della macchina astratta dipenderà, quindi, da quelle funzionalità che vengono messe a disposizione dei livelli sottostanti.

Realizzare M_L consiste nel realizzare una macchina che traduce L in L_0 , ovvero che interpreta tutte le istruzioni di L come (sequenza di) istruzioni di L_0 .

Si possono distinguere due modalità:

- ◊ **traduzione esplicita** (soluzione compilativa): macchina che traduce programmi scritti in L in programmi scritti in L

$$P_L \xrightarrow{\text{traduzione}} P_{L_0} \xrightarrow{\text{esecuzione}} M_{L_0}$$

- ◇ **traduzione implicita** (soluzione interpretativa): macchina simula ogni istruzione di programmi in L mediante programmi/istruzioni scritti in L_0

$$P_L \xrightarrow{\text{simulazione}} M_{L_0}$$

1.7.1 Soluzione interpretativa

La soluzione interpretativa è una traduzione esplicita, ovvero consiste nella simulazione di L in M_{L_0} .

Notazione

- **Prog^L**: insieme programmi scritti in L
- **D**: insieme dati (input e output)
- $P^L \in \text{Prog}^L$ e $\text{in}, \text{out} \in D$
- $\llbracket P^L \rrbracket: D \rightarrow D$ semantica di P^L tale che $\llbracket P^L \rrbracket(\text{in}) = \text{out}$ se P^L eseguito a partire da in restituisce out

Dato P^L su dati D , un interprete $I^{L_0,L}$ è un programma che riceve in input $P^L \in \text{Prog}^L$ e $d \in D$.

Definizione: interprete

Un **interprete** per L , scritto in L_0 , è un programma che realizza la funzione:

$$\underbrace{I^{L_0,L} : (\text{Prog}^L \times D) \rightarrow D}_{I^{L_0,L}(P^L, \text{input}) = P^L(\text{input}) \quad \circ \quad \llbracket I^{L_0,L} \rrbracket(P^L, \text{input}) = \llbracket P^L \rrbracket(\text{input})}$$

$\Rightarrow$ l'interprete applicato a P^L e al dato (input) restituisce esattamente ciò che restituirebbe P^L applicato all'input.

L'interprete, quindi, è un altro programma che prende P^L e lo esegue passo passo.

Questo oggetto (interprete) viene definito **macchina universale**.

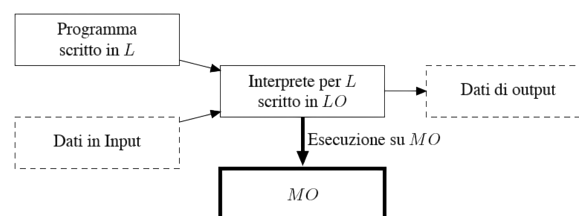


Figura 1.3: schema grafico dell'interprete

1.7.1.1 Operazioni

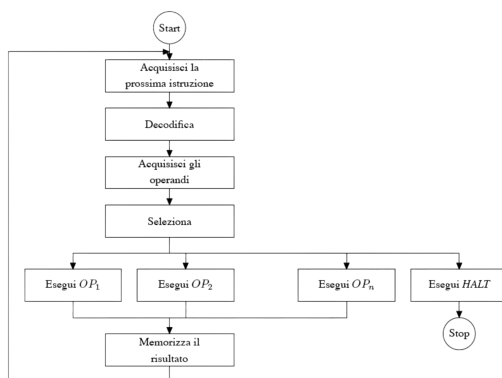
Un interprete è di fatto un ciclo che attraverso una serie di operazioni e funzionalità esegue per simulazione le istruzioni del linguaggio.

Le operazioni principali sono:

- ▷ **elaborazione dei dati primitivi** (dati rappresentati in modo diretto nella memoria)
- ▷ **controllo della sequenza di esecuzione**: gestione del flusso
- ▷ **controllo dei dati**: recupero dati per l'esecuzione delle istruzioni
- controllo della memoria**: gestione dell'allocazione dei dati e programmi

1.7.1.2 Struttura

Stabilite le operazioni di cui un interprete ha bisogno, vediamo come opera concretamente. Il ciclo di esecuzione comune a tutti gli interpreti è:



(a) flow-chart

```

begin
  go := true;
  while go do begin
    FETCH(OPCODE, OPINFO) at PC
    DECODE(OPCODE, OPINFO)
    if OPCODE needs ARGS then FETCH(ARGS)
    case OPCODE of
      OP1: EXECUTE(OP1, ARGS)
      ...
      OPn: EXECUTE(OPn, ARGS)
      HLT: go := false;
    if OPCODE has result then STORE(RES);
    PC := PC + SIZE(OPCODE);
  end
end
  
```

Controllo dati (CD) points to the `if OPCODE needs ARGS then FETCH(ARGS)` line.

Controllo sequenza (CS) points to the `while go do begin` and `HLT: go := false;` lines.

(b) pseudocodice

1.7.2 Soluzione compilativa

La soluzione compilativa è una traduzione implicita.

Notazione

- Prog^L : insieme programmi scritti in L
- D : insieme dati (input e output)
- $P^L \in \text{Prog}^L$ e $\text{in}, \text{out} \in D$
- $\llbracket P^L \rrbracket: D \rightarrow D$ semantica di P^L tale che $\llbracket P^L \rrbracket(\text{in}) = \text{out}$ se P^L eseguito a partire da in restituisce out

Costruiamo $C^{L_0, L}$ dove L è il linguaggio da implementare (linguaggio sorgente) e L_0 è il linguaggio macchina ospite (linguaggio target).

Definizione: compilatore

Un **compilatore** da L a L_0 è un programma che realizza la funzione:

$$C^{L_0, L}: \text{Prog}^L \rightarrow \text{Prog}^{L_0}, p^L \in \text{Prog}^L$$

$$C^{L_0, L}(P^L) = P^{L_0} \mid \forall \text{in}: P^L(\text{in}) = P^{L_0}(\text{in}) \circ \llbracket C^{L_0, L} \rrbracket(P^L) = P^{L_0} \mid \forall \text{in}: \llbracket P^L \rrbracket(\text{in}) = \llbracket P^{L_0} \rrbracket(\text{in})$$

In questo caso, quindi, la fase di traduzione viene separata dalla fase di esecuzione.

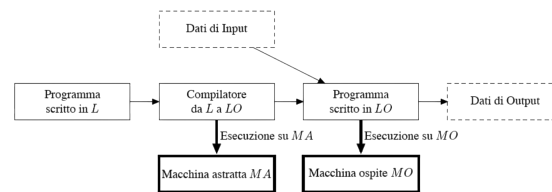


Figura 1.5: schema grafico compilatore

1.7.2.1 Struttura

L'esecuzione di un compilatore passa attraverso varie fasi:

- ▷ **analisi lessicale** (scanner): individua le unità lessicali (variabili, comandi, valori, ecc.)
- ▷ **analisi sintattica** (parser): analizza se gli elementi lessicali sono stati costruiti nel modo corretto mediante l'uso di parse-tree
- ▷ **analisi semantica + generazione codice intermedio**
- ▷ **generazione codice**

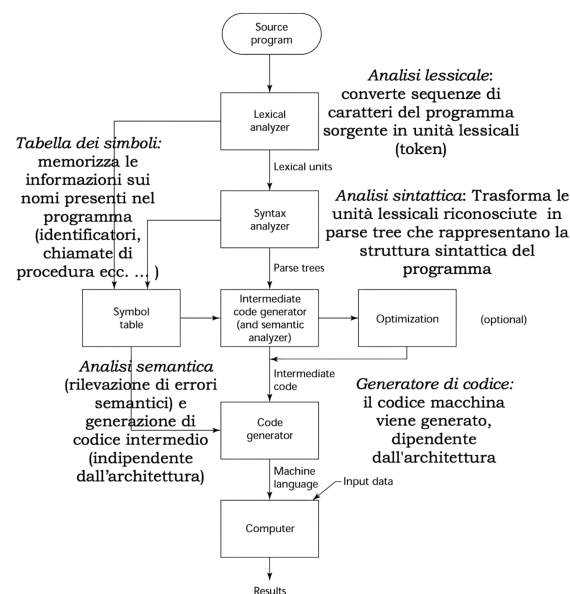
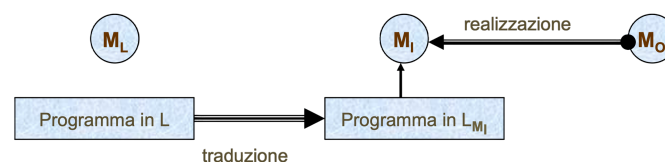


Figura 1.6: flow-chart

1.7.3 Soluzione ibrida

Esiste un compromesso tra compilatore e interprete, ovvero una soluzione ibrida dove il linguaggio ad alto livello viene compilato in un linguaggio a più basso livello che poi viene interpretato.



In questo caso consideriamo il linguaggio L , ad alto livello, per il quale dobbiamo realizzare la macchina astratta M_L .

L viene, quindi, tradotto in un linguaggio intermedio L_{M_i} la cui macchina astratta M_L consiste in un interprete del linguaggio L_{M_i} sulla macchina ospite M_0 .

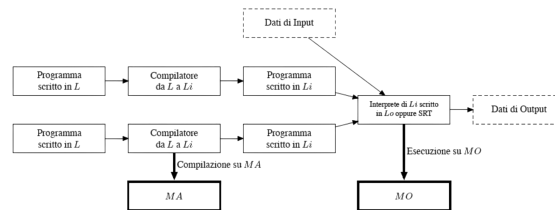


Figura 1.7: schema grafico soluzione ibrida

Esempio: linguaggi di programmazione con soluzione ibrida

Un linguaggio di programmazione che utilizza la soluzione ibrida è Java.

Descrivere i linguaggi di programmazione

Un linguaggio di programmazione è un formalismo artificiale utilizzato per esprimere algoritmi. Anche se artificiale, è sempre un linguaggio e, quindi, deve essere descritto.

Un linguaggio di programmazione non è altro che come un linguaggio naturale, ma semplificato e non ambiguo.

La descrizione di un linguaggio PL avviene tramite:

- ▷ **sintassi** (grammatica): regole di formazione di frasi ben formate (descrive relazione tra segni)
- ▷ **semantica**: attribuzione di significato ai simboli della sintassi (costruisce relazione tra segni e significato)
- ▷ **pragmatica**: modo in cui le frasi del linguaggio di programmazione vengono usate
- ▷ **implementazione**: strumenti per eseguire frasi del linguaggio di programmazione rispettandone la semantica

2.1 Sintassi

Definizione: sintassi

Le **regole sintattiche** del linguaggio specificano quali stringhe di caratteri sono legali nel linguaggio.

La terminologia nella linguistica sono:

- **parola**: sequenza (stringa) di caratteri in un alfabeto
- **frase**: sequenza (ben formata) di parole
- **linguaggio**: insieme di frasi

Nell'ambito dei linguaggi di programmazione, gli stessi nomi prendono nomi differenti:

- ◊ **lessema** (parola): parola (terminale) con significato proprio.
Corrisponde all'**unità sintattica a più basso livello dei PL**.

- ◇ **token** (frase): sequenze di simboli non terminali.
Corrispondono agli **elementi delle categorie sintattiche**.

2.1.1 Descrivere la sintassi

Abbiamo bisogno di strumenti formali per descrivere parole e frasi ben formate di un linguaggio, che permettono, in modo automatico, di verificare l'appartenenza o meno al linguaggio.

Definizione: riconoscitore

Il **riconoscitore** è uno strumento (formale) di riconoscimento che legge in input stringhe sull'alfabeto e decide se la stringa appartiene o meno al linguaggio.

Es. automi a stati finiti (analisi sintattica)

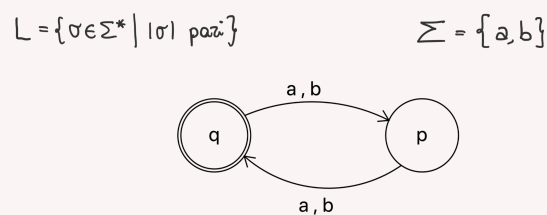


Figura 2.1: esempio di automa a stati finiti

Definizione: generatore

Il **generatore** è uno strumento (formale) che genera stringhe di un linguaggio.

Es. grammatiche CF (parser)

$$S := \varepsilon \mid aaS \mid abS \mid baS \mid bbs$$

Figura 2.2: esempio di stringhe generate da un automa a stati finiti

2.1.2 Grammatiche CF

La grammatica descrive i modi per combinare parole e formare frasi ed è descritta da:

- ▷ **alfabeto**
- ▷ **descrizione lessicale:** descrizione delle parole
- ▷ **descrizione delle frasi**

Esempio: grammatica

$$L = \{\sigma \in \Sigma^* \mid \sigma \text{ palindromo}\}$$

Def: • ε , a e b sono palindromi
 • se σ è palindromo allora lo
 sono anche $a\sigma a$ e $b\sigma b$

$$S := \varepsilon \mid a \mid b \mid aSa \mid bSb$$

Per costruire una grammatica abbiamo bisogno di:

- **vocabolario**
- **regole di composizione**: come combinare le parole (produzioni) per ottenere frasi ben formate

Le grammatiche CF permettono di descrivere la sintassi dei linguaggi di programmazione. Questo generatore di linguaggi è stato sviluppato da Chomsky per descrivere la sintassi dei linguaggi naturali e permette di definire la classe di linguaggi CF.

Definizione: grammatica CF

Una **grammatica CF** è una quadrupla

$$G = \langle V, T, P, S \rangle$$

dove:

- ◊ **V**: insieme dei simboli non terminali
- ◊ **T**: insieme dei simboli terminali ($V \cup T = \emptyset$)
- ◊ **P**: insieme delle produzioni

$$\underbrace{A}_{\in V} \rightarrow \alpha \in (V \cup T)^*$$

a un simbolo non terminale A possiamo sostituire (indipendente dal contesto) la sequenza α di simboli terminali e non

- ◊ $\underbrace{S \in V}$: simbolo iniziale
 rappresenta
 il linguaggio
 da generare

Nella grammatica i simboli terminali costituiscono il vocabolario (collezione di parole/token), invece, i simboli non terminali sono le categorie sintattiche (espressioni, comandi, dichiarazioni, ecc.).

Le produzioni sono le regole di formazione/composizione (istruzioni ... programmi).

Il linguaggio è l'insieme di tutte le frasi (stringhe di simboli terminali) generate a partire dal simbolo iniziale S.

2.1.2.1 Notazione BNF

La notazione BNF viene ideata nel 1959 da Bakus per Algol60.

Questa notazione è usata per descrivere grammatiche CF, dove si usano intere parole come simboli terminali, mentre i non terminali sono identificati racchiudendoli tra parentesi angolate <>.

Per le produzioni, invece, viene utilizzato il simbolo $::=$ al posto della freccia $\rightarrow$.

$$\begin{aligned} \langle A \rangle & ::= \alpha \langle B \rangle \gamma \mid \alpha \\ \langle B \rangle & ::= \varepsilon \mid \beta_1 \mid \beta_2 \end{aligned}$$

Figura 2.3: esempio di notazione BNF

2.1.3 Descrivere un semplice linguaggio imperativo

Un linguaggio può essere descritto anche in modo informale descrivendo quali caratteristiche deve avere qual è il loro significato:

- ▷ niente dichiarazioni
- ▷ solo variabili ed espressioni booleane
- ▷ assegnamento e composizione sequenziale
- ▷ comando condizionale
- ▷ comando imperativo (loop)

Anche i linguaggi di programmazione sono descritti a livelli di astrazione, ognuno descritto da una grammatica.

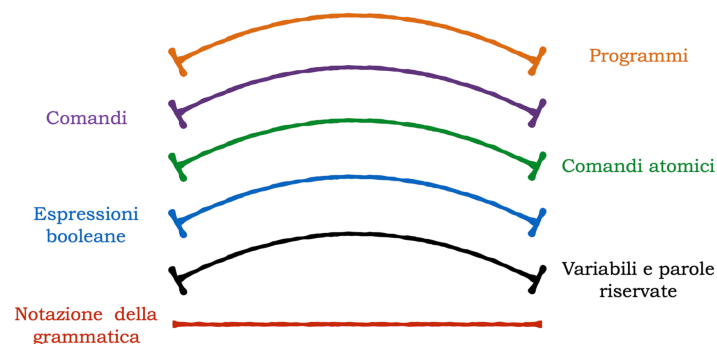


Figura 2.4: livelli di astrazione dei PL

Notazione grammatica

```
1 <bExpr> B -> true | false | v | (not B) | (B and B) | (B or B)
2
3 <atomic com> A -> v := B
4
5 <com> C -> A | C ; C | if B then C else C | while B do C
6
7 <program> S -> C
```

2.1.3.1 Analisi semantica

Esistono dei vincoli che la grammatica CF non può catturare:

- ▷ **vincoli contestuali**: dichiarare una variabile prima dell'uso, n° di parametri di una chiamata

Il problema è che non esistono algoritmi per il riconoscimento di stringhe generate, quindi, non ci sono algoritmi per fare il parser di queste stringhe.

Per questo la scelta ricade su una specifica del linguaggio mediante grammatica CF (sintassi) e poi, come parte della semantica (semantica statica) vengono specificati i vincoli contestuali.

2.1.4 Categorie sintattiche**Definizione: categorie sintattiche**

Le **categorie sintattiche** sono classi di elementi con diverso significato.

Le categorie sintattiche sono, quindi, gli elementi non terminali della grammatica il cui significato dipende da ciò che denotano, manipolano e producono.

In particolare, ci sono categorie sintattiche che permettono di:

- ◇ manipolare valori
- ◇ creare/gestire i legami
- ◇ eseguire trasformazioni di stato

Definizione: stato

Il concetto di **stato** è composto da due concetti:

- ▷ **ambiente** (environment): insieme dei legami (bindings) tra identificatori e oggetti identificati
- ▷ **memoria** (store): insieme degli effetti sugli identificatori

In generale, le categorie sintattiche non si devono sovrapporre.

2.1.4.1 Espressioni

Definizione: espressioni

Le **espressioni** denotano valori.

Le espressioni devono essere valutate per restituire un valore.

Due espressioni possono essere valutate diversamente, ma possono denotare lo stesso valore.

$$\text{not } (a \text{ and } b) \equiv (\text{not } a) \text{ and } (\text{not } b)$$

Queste due espressioni sono semanticamente equivalenti, ma sono sintatticamente diverse.

Definizione: equivalenza tra espressioni

Due espressioni si dicono equivalenti se denotano lo stesso valore in ogni stato.

2.1.4.2 Comandi

Definizione: comandi

Le **comandi** denotano richieste di trasformazioni/modifiche della memoria.

I comandi, quindi, rappresentano funzioni di trasformazione (da stato a stato) e devono essere eseguiti per poter attuare la trasformazione (irreversibile) della memoria.

Definizione: equivalenza tra comandi

Due comandi si dicono equivalenti se per ogni stato (memoria) in input producono lo stesso stato (memoria) in output.

2.1.4.3 Dichiarazioni

Definizione: dichiarazioni

Le **dichiarazioni** denotano richieste di creazione/modifiche di ambienti.

Le dichiarazioni devono essere elaborate per attuare la creazione/modifica dell'ambiente, ovvero la modifica (reversibile) dei legami.

Reversibile

Reversibile = vale esclusivamente nel raggio di azione (scope) dell'identificate.

Definizione: equivalenza tra dichiarazioni

Due dichiarazioni si dicono equivalenti se producono lo stesso ambiente.

2.1.5 Descrivere un semplice linguaggio implementato

Il seguente linguaggio è un esempio di sintassi per il linguaggio di programmazione PL/0 simile al linguaggio Pascal in cui:

- ▷ un solo tipo di valore INTEGER
- ▷ strutture di controllo classiche
- ▷ astrazione del controllo (procedure)

Comandi del linguaggio PL/0

```

1  <program>    P -> B
2
3  <block>      B -> D S
4
5  <declar>     D -> [const I=N{,I=N};] | [var I{,I};] |
6                [procedure I;B]
7
8  <stmts>      S -> ep | I := E | call I | if C then S |
9                while C do S | begin S{;S} end
10
11 <Cond>       C -> odd E | E > E | E >= E | E = E | E # E |
12               E < E | E <= E
13
14 <Expr>       E -> I | N | E bop E | (E)

```

2.1.5.1 Sintassi

I programmi P sono blocchi B.

I blocchi B sono una dichiarazione D seguita da un comando S.

Nota

D ed S sono definiti in modo libero dal contesto, ovvero quello che possiamo generare da S non deve avere necessariamente nessun vincolo con quello che generiamo da D.

Le dichiarazioni D sono dichiarazioni di valori costanti con nome (const), dichiarazioni di variabili (var) e dichiarazioni di procedure (ovvero di un blocco riferibile con un nome).

I nomi sono gli identificatori I, considerati qui parte del vocabolario (N sono i numeri naturali).

Non abbiamo bisogno del concetto di tipo in quanto abbiamo solo un tipo di dato.

Si noti che le produzioni di D sono tutte opzionali, questo significa che un programma può essere un blocco senza dichiarazioni.

I comandi sono:

- ◇ comando vuoto

- ◇ assegnamento di un valore (denotato da una espressione) ad un identificatore
- ◇ chiamata ad una procedura mediante il suo nome
- ◇ comando condizionale che esegue un comando al verificarsi di una condizione booleana C
- ◇ ciclo che ripete un comando finché una data condizione C è vera
- ◇ composizione sequenziale di comandi

2.2 Semantica

La semantica è più complessa della sintassi perchè ricerca:

- ▷ **esattezza**: descrizione precisa e non ambigua di cosa ci dobbiamo aspettare quando eseguiamo un programma
- ▷ **flessibilità**: non deve anticipare scelte legate all'implementazione

Definizione: semantica

La **semantica** descrive il significato dei programmi.

⇒ **significato** = entità autonoma che esiste indipendentemente dai segni a cui viene associato

Non esiste un formalismo univoco per rappresentare la semantica, ma esistono diverse metodologie che si adattano alle necessità.

Non è possibile, però, dare un significato a tutti i programmi perchè sono infiniti e, quindi, anche la semantica deve fornire una rappresentazione finita del significato dei programmi sfruttando la loro procedura di costruzione (sintassi) che genera in modo induttivo, sulle regole della grammatica, tutti i possibili programmi.

2.2.1 Induzione matematica

Definizione: induzione matematica

Dato un insieme A e una relazione binaria $< \subseteq A \times A$ ben fondato (senza catene discendenti infinite).

Se $A = \mathbb{N}$ si ha **induzione matematica**.

L'induzione (matematica) è un principio usato per definire e dimostrare.

Principio di induzione (matematica)

Π è la proprietà che vogliamo dimostrare sugli elementi di A .

$$\forall a \in \underbrace{A}_{\Pi \text{ vale su } a \in A} \Leftrightarrow \forall a \in A [[\forall b < a. \Pi(b)] \Rightarrow \Pi(a)]$$

Per eseplicitare il vantaggio di avere A ben fondato, seperiamo la dimostrazione sugli elementi minimali (base):

$$\text{base} = \{a \in A \mid a \text{ minimale in } A\} \quad (\text{base è finita})$$

$$\underbrace{\forall a \in \text{base}_A. \Pi(a)}_{\text{base induttiva}} \wedge \forall a \in A, \text{base}_A. \underbrace{[\forall b < a. \Pi(b)]}_{\text{hp induttiva}} \xRightarrow{\text{passo induttivo}} \Pi(a)$$

2.2.2 Induzione strutturale

Definizione: induzione strutturale

Dato un insieme A e una relazione binaria $< \subseteq A \times A$ ben fondato (senza catene discendenti infinite).

Se $A = \underbrace{L(G)}_{\text{linguaggio generato da } G}$ si ha **induzione strutturale** (G è grammatica).

L'induzione strutturale permette di dimostrare che una proprietà vale su tutti gli oggetti di una categoria sintattica (sulla grammatica).

Principio induzione strutturale

1. dimostrazione/definizione della proprietà su tutti gli elementi base della categoria (**base**)
2. dimostrazione/definizione della proprietà su tutti gli elementi composti, assumendo la proprietà vera/definita per tutti i componenti immediati (**hp induttiva**)

2.2.3 Proprietà della semantica

Le proprietà fondamentali di qualunque semantica sono:

- **composizionalità**: significato di ogni programma è funzione del significato dei costituenti immediati
- **equivalenza**: due programmi sono equivalenti quando hanno lo stesso significato (semantica).

L'equivalenza è necessaria in varie fasi di analisi:

- **correttezza**: dimostrare che il programma scritto calcola esattamente la funzione attesa
- **equivalenza di programmi**: dimostrare che due programmi calcolano la stessa funzione
- **efficienza**: dati due programmi che calcolano la stessa funzione, dimostrare quale lo fa in modo più efficiente

2.2.4 Tipi di semantica

Un algoritmo è un'entità astratta concretizzata in un programma, ma non ci interessa sapere se il programma implementa esattamente l'algoritmo, ma ci interessa che il programma implementi la funzione descritta dall'algoritmo.

A seconda di quali strumenti matematici vengono usati si ottengono tipi di semantiche diverse rendendo più agevoli tipi diversi di ragionamento.

- ▷ **semantica denotazionale**: descrive funzionalità (es. studia gli effetti dell'esecuzione ecc.)
- ▷ **semantica operativa**: descrive trasformazioni di stato (guarda come i risultati finali vengono prodotti)
- ▷ **semantica assiomatica**: descrive proprietà soddisfatte dall'esecuzione del programma

2.2.4.1 Semantica denotazionale

Definizione: semantica denotazionale

La **semantica denotazionale** è un modello matematico di programmi basato sulla ricorsione.

Il processo di costruzione della semantica denotazionale per un linguaggio consiste nel definire:

- ◇ un oggetto matematico (denotazione) che rappresenta ogni elemento del linguaggio
- ◇ l'associazione tra elemento e denotazione

Formalmente, il modello usato per descrivere la semantica è quello delle funzioni matematiche (ricorsive), ovvero un programma corrispondente ad una funzione tra stati della macchina.

$$\mathcal{E} : \text{Prog} \rightarrow ((\text{Var} \rightarrow \text{Val}) \rightarrow (\text{Var} \rightarrow \text{Val}))$$

La semantica denotazionale, quindi, è un oggetto puramente matematico che descrive gli effetti manipolando oggetti matematici.

In particolare, guarda agli effetti dell'esecuzione del programma, descrivendo l'effetto dell'esecuzione di una sequenza di comandi separati da ; attraverso la composizione degli effetti dei singoli comandi da sinistra verso destra.

Analisi di un programma

Programma semplice

```

1      P :
2          z := 2;
3          y := z;
4          y := y+1;
5          z := y;
```

$$\begin{aligned}
\mathcal{E}[P][z=\perp, y=\perp] = & (\mathcal{E}[z:=y] \circ \mathcal{E}[y:=y+1] \circ \mathcal{E}[y:=z] \circ \mathcal{E}[z:=2])[z=\perp, y=\perp] = \\
& (\mathcal{E}[z:=y](\mathcal{E}[y:=y+1](\mathcal{E}[y:=z](\mathcal{E}[z:=2][z=\perp, y=\perp]))) = \\
& \quad \quad \quad [z=2, y=\perp] \\
& \quad \quad \quad [z=2, y=2] \\
& \quad \quad [z=2, y=3] \\
& [z=3, y=3]
\end{aligned}$$

Questa semantica può essere usata per dimostrare la correttezza dei programmi, ma a causa della sua complessità ha poca utilità per gli utilizzatori dei linguaggi.

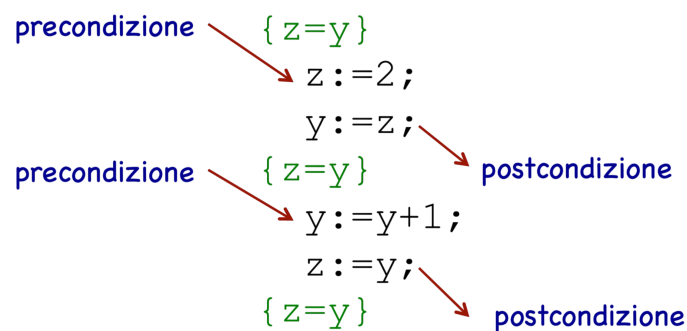
2.2.4.2 Semantica assiomatica

Definizione: semantica assiomatica

La **semantica assiomatica** è un modello matematico dei programmi basato sulla logica formale (calcolo dei predicati).

La semantica assiomatica nasce con l'obiettivo di fare verifica formale di programmi.

In particolare permette di dimostrare proprietà parziali di correttezza: quando lo stato iniziale rispetta la precondizione e il programma termina, allora lo stato finale soddisfa la postcondizione.



Analisi di un programma

Programma semplice

```

1      P :
2          z := 2;
3          y := z;
4          y := y+1;
5          z := y;

```

$$\frac{\{P\} S \{Q\}, P' \Rightarrow P, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

$$\frac{\{P1\} S1 \{P2\}, \{P2\} S2 \{P3\}}{\{P1\} S1; S2 \{P3\}}$$

$$\frac{\{B \text{ and } P\} S1 \{Q\}, \{\text{(not } B) \text{ and } P\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$

$$\frac{(I \text{ and } B) S \{I\}}{\{I\} \text{ while } B \text{ do } S \{I \text{ and (not } B)\}}$$

2.2.4.3 Semantica operativa

Definizione: semantica operativa

La **semantica operativa** è un modello matematico dei programmi basato sui sistemi di transizione.

La semantica operativa descrive il significato del programma eseguendo i suoi comandi su una macchina.

L'evoluzione dello stato della macchina definisce il significato del comando.

- ▷ **memoria**: $\sigma = \text{Var} \rightarrow \text{Val}$
- ▷ **stato**: $\langle P, \sigma \rangle$ (programma - memoria)

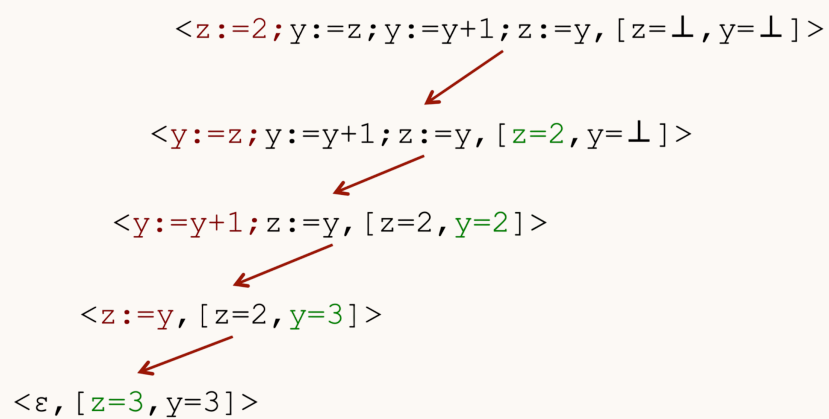
Ad ogni passo di esecuzione descriviamo un cambio di stato: $\rightarrow \subseteq \langle P, \sigma \rangle \times \langle P, \sigma \rangle$ (relazione di transizione).

La **chiusura transitiva** descrive l'esecuzione completa.

Analisi di un programma

Programma semplice

```
1      P :  
2          z := 2;  
3          y := z;  
4          y := y+1;  
5          z := y;
```



Espressioni

Le **espressioni** sono la categoria sintattica il cui significato è un valore ottenuto mediante una valutazione.

3.1 Descrivere le espressioni

Il valore ottenuto mediante la valutazione di un'espressione è detto **valore esprimibile**, in quanto è un valore che può essere espresso attraverso la sintassi dei linguaggi di programmazione (sintassi delle espressioni).

Definizione: valore esprimibile

Un **valore esprimibile** è definito come:

$$\text{Eval} = \underbrace{\text{int}}_{\mathbb{Z}} \cup \underbrace{\text{bool}}_{\{\text{true}, \text{false}\}}$$

Gli aspetti che caratterizzano le espressioni sono:

- ▷ **arietà degli operatori e notazione:**
- ▷ **regole di precedenza** (tra operatori)
- ▷ **regole di associa** (degli operatori)
- ▷ **ordine di valutazione degli operandi**
- ▷ **presenza di side effect**
- ▷ **overloading degli operatori**
- ▷ **espressioni con tipi misti**

3.1.1 Notazione

Le espressioni possono essere denotate in modo diversi:

- **notazione in-fissa:** $a + b$
- **notazione pre-fissa:** $+ a b$
- **notazione post-fissa:** $a b +$

La scelta della notazione può determinare le regole di associatività e/o precedenza, o incidere su altre caratteristiche delle espressioni.

3.1.1.1 Notazione in-fissa

La **notazione in-fissa** è quella più usata in matematica.

Questa notazione, infatti, è quella che permette una lettura più intuitiva, ma richiede maggiori specifiche.

Per prima cosa, però, è necessario stabilire le regole di precedenza e associatività.

Inoltre, è necessario l'utilizzo delle parentesi per permettere precedenze o associatività diverse da quelle regole o aumentare la leggibilità.

$a + b * c ** d ** e / f \quad ??$

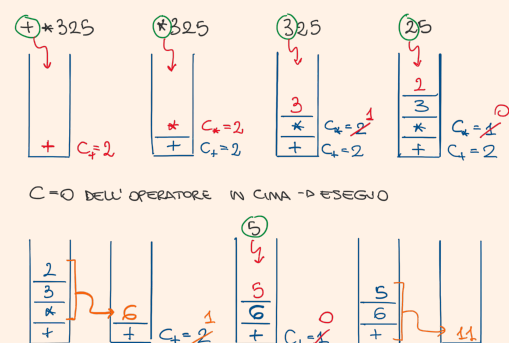
if A < B and C < D then ??
(in Pascal Errore se A,B,C,D
non sono tutti booleani)

3.1.1.2 Notazione pre-fissa

La **notazione pre-fissa** è più semplice dell'in-fissa in quanto non necessita di regole di precedenza e associatività, e non necessita nemmeno di parentesi.

Algoritmo di valutazione

1. leggere il prossimo simbolo dell'espressione e metterlo nella pila
2. se il simbolo è un operatore op: $Cop = n$ e torna a 1
3. se il simbolo letto è un operando inserirlo nella pila e calcola $Cop = Cop - 1$ (op ultimo operatore inserito)
4. se $Cop \neq 0$ torna a 1 (op ultimo operatore inserito)
5. se $Cop = 0$ allora:
 - applicare l'ultimo operatore op agli operandi in cima alla pila (che vanno cancellati) e caricare il risultato nella pila e se non ci sono più simboli vai a 6
 - $Cop = n - m$ (n arietà del nuovo ultimo operatore op nella pila, m numero di operandi sopra l'operatore nella pila)
 - tornare a 4
6. se la pila non è vuota tornare a 1

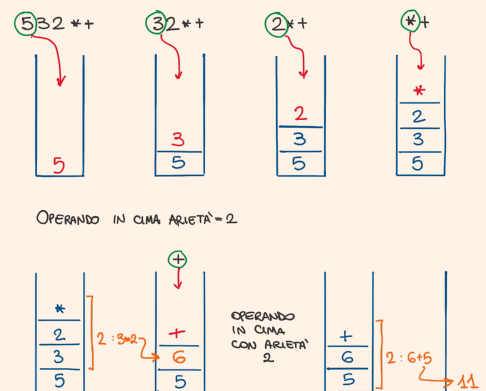


3.1.1.3 Notazione post-fissa

La **notazione post-fissa** è più semplice dell'in-fissa in quanto non necessita di regole di precedenza e associatività, e non necessita nemmeno di parentesi.

Algoritmo di valutazione

1. leggere il prossimo simbolo dell'espressione
 - ▷ se il simbolo è un operatore (di varietà n)
 - applicare l'operatore a n operandi immediatamente precedenti sulla pila
 - memorizzare il risultato in R
 - eliminare operatore e operandi dalla pila
 - memorizzare il valore di R sulla pila
 - ▷ se il simbolo letto è un operando inserito nella pila
2. se ci sono ancora simboli nell'espressione da leggere tornare a 1 altrimenti terminare



3.1.2 Regole di precedenza

Definizione: regole di precedenza

Le **regole di precedenza** definiscono l'ordine in cui operatori "adiacenti" a diversi livelli di precedenza vengono valutati.

Tipici livelli di precedenza:

- ▷ parentesi
- ▷ operatori unitari
- ▷ $**$ (se esistono nel linguaggio)
- ▷ $*$, $/$
- ▷ $+$, $-$

3.1.3 Regole di associatività

Definizione: regole di associatività

Le **regole di associatività** stabiliscono l'ordine con cui operatori allo stesso livello di precedenza vengono valutati.

Tipiche regole di associatività:

- ▷ da dx a sx
- ▷ operatori associano a dx (es. FORTRAN)
- ▷ parentesi sovrasrivono le regole

3.1.4 Valutazione delle espressioni

La rappresentazione interna delle espressioni consiste in una rappresentazione ad albero. L'albero fornisce la struttura dell'espressione, ovvero elimina eventuali ambiguità legati a precedenze e associatività, ma non può dare informazioni riguardanti l'ordine di valutazione dell'espressione in quanto ci sono vari aspetti che possono influire sul risultato:

- ▷ **operandi non definiti**: la presenza di operandi non definiti può far sì che l'espressione risultante possa essere valutata solo con un preciso ordine di valutazione.

Esempio: operandi non definiti

Consideriamo questa espressione:

$$a == 0 ? b : b/a$$

Quando $a = 0$ l'espressione dà errore perché b/a non è definita. È vero, anche, che la valutazione di b/a avviene solo quando a è diverso da 0.

A seconda del tipo di considerazioni possiamo avere:

- ⇒ **valutazione eager**: vengono valutati sempre tutti gli operandi coinvolti
- ⇒ **valutazione lazy**: gli operandi vengono valutati solo quando è necessario
- ▷ **effetti collaterali** (side effect): sono effetti non propriamente legati all'oggetto che si sta manipolando (es. valutazione di un'espressione causa cambiamento della memoria o quando una funzione modifica i propri parametri)

Esempio: effetti collaterali

```
a = 10;
b = a + fun(a)
           modifica valore di a per l'intero programma
```

- ▷ **aritmetica finita**: nelle macchine l'aritmetica è limitata dall'architettura, quindi, i numeri non sono infiniti ed esiste un massimo numero rappresentabile. Il problema di valutazione si ha in caso di espressioni che coinvolgono tale massimo numero. Tipicamente l'ordine di valutazione risolve il problema per prima cosa le variabili e le costanti poi valuta gli operandi seguendo la struttura data dalle parentesi dell'espressione.

3.2 Sintassi

La sintassi delle espressioni è la seguente:

Sintassi delle espressioni

```

1   <Exp>
2   E -> A | B
3   A -> N | A op A
4   B -> true | false | not B | B or B | A = A

```

dove:

- ▷ A: espressioni aritmetiche
- ▷ B: espressioni booleane
- ▷ N: simboli che rappresentano numeri
- ▷ op: operatore numerico ($op \in \{+, -, *\}$)

3.3 Semantica

Definiamo il sistema di transizione che determina il processo di valutazione delle espressioni. Per definire un sistema di transizione di valutazione, abbiamo bisogno di definire:

- ▷ **configurazioni**: definite come $\Gamma = \epsilon$, ovvero le espressioni generate dalla grammatica. Il sottoinsieme $T \subseteq \Gamma$ terminali sono $T = \mathcal{N}$ (numerali della macchina su cui eseguiamo)
- ▷ **regole di transizione**: determina i passaggi di valutazione delle espressioni:
 - ◊ espressioni aritmetiche: $op \in \{+, -, *\}$
 - ◊ espressioni booleane: $bop \in \{or, +, -, *\}$

3.3.1 Regole di transizione: espressioni aritmetiche

Le regole di transizione per le espressioni sono:

- **regola 1** (assioma): definisce p come la semantica del calcolo effettivo di un operatore

regola $\mathcal{E}_1$ (assioma)

$$\mathcal{E}_1 : \underbrace{m \text{ op } n}_{m, n, p \in \mathcal{N}} \rightarrow p \quad \Leftrightarrow \quad m \text{ op } n = p$$

- **regola 2**: definisce la valutazione dell'espressione a sinistra per prima

regola $\mathcal{E}_2$

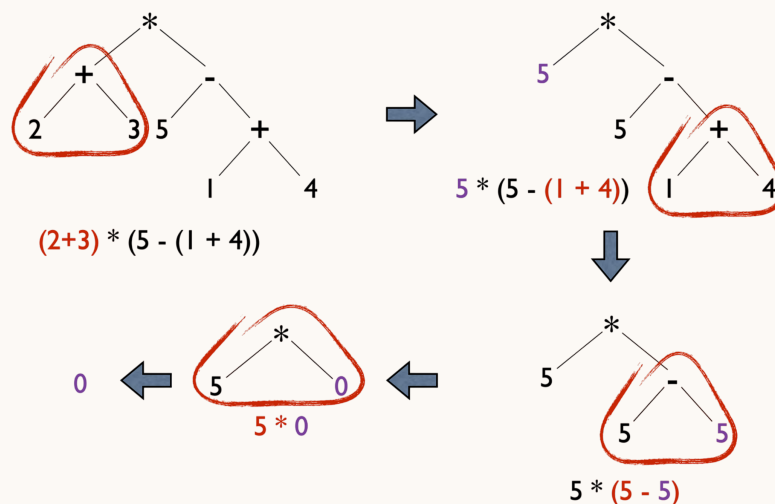
$$\mathcal{E}_2 : \frac{e \rightarrow e'}{e \text{ op } e_0 \rightarrow e' \text{ op } e_0}$$

- **regola 3**: definisce la valutazione dell'espressione a destra con un'espressione già valutata a sinistra

regola $\mathcal{E}_3$

$$\mathcal{E}_3 : \frac{e \rightarrow e'}{m \text{ op } e \rightarrow m \text{ op } e'}$$

Esempio: regole di transizione



3.3.1.1 Regole di transizione: espressioni booleane

Le regole di transizione per le espressioni sono:

- **regola 4** (assioma):

regola $\mathcal{E}_4$ (assioma)

$$\mathcal{E}_4 : \underbrace{k_1}_{\in \mathcal{N} \cup \mathcal{B}} \text{ bop } \underbrace{k_2}_{\in \mathcal{N} \cup \mathcal{B}} \rightarrow \underbrace{t}_{\in \mathcal{B}} \Leftrightarrow k_1 \text{ bop } k_2 = t$$

- **regola 2'**:

regola $\mathcal{E}'_2$

$$\mathcal{E}'_2 : \frac{e \rightarrow e'}{e \text{ bop } e_0 \rightarrow e' \text{ bop } e_0}$$

- **regola 3'**:

regola $\mathcal{E}'_3$

$$\mathcal{E}'_3 : \frac{e \rightarrow e'}{k \text{ bop } e \rightarrow k \text{ bop } e'}$$

◦ **regola 5** (assioma):

regola $\mathcal{E}_5$

$$\mathcal{E}_5 : \text{not } \underbrace{t_0}_{\in \mathcal{B}} \rightarrow \underbrace{t}_{\in \mathcal{B}} \Leftrightarrow \text{not } t_0 = t$$

◦ **regola 6**:

regola $\mathcal{E}_6$

$$\mathcal{E}_6 : \frac{e \rightarrow e'}{\text{not } e \rightarrow \text{not } e'}$$

Con queste regole di possono valutare ogni tipo di espressione.

Definizione: valutazione

Il processo di **valutazione** è una funzione:

$$\text{Eval} : \mathcal{E} \rightarrow \mathcal{N} \cup \mathcal{B} \mid \text{Eval}(e) = k \in \mathcal{N} \cup \mathcal{B} \Leftrightarrow e \rightarrow *K$$

La valutazione di e è k se e solo se nel sistema di transizione con un certo numero di transizione possono transire da e a k .

Definizione: equivalenza

La **relazione di equivalenza** è definita come:

$$\equiv \subseteq \mathcal{E}_1 \times \mathcal{E}_2$$

allora:

$$e_1 \equiv e_2 \Leftrightarrow \text{Eval}(e_1) = \text{Eval}(e_2)$$

Identificatori

Definizione: identificatore

Un **identificatore** è una sequenza alfa-numerica di caratteri usate per denotare e riferire oggetti di varia natura.

```
const pi = 3,14           //denota un valore
int x                     //denota una variabile
void f(){...}             //definisce una funzione (procedura)
```

Bisogna ricordare che:

identificatore $\neq$ oggetto denotato

Infatti, è possibile avere più identificatori che si riferiscono allo stesso oggetto (aliasing).

E' possibile, anche che più oggetti denotati dallo stesso identificatori (polimorfismo).

I linguaggi di programmazione possono porre delle regole (limiti) a quello che può essere usato come identificatore:

- ▷ **sono oppure no case sensitive?**

in C pippo, Pippo, PIPPO sono identificatori diversi

- ▷ **esistono parole riservate?** (parole già associate a specifici significati)
- ▷ **esistono restrizioni sulla lunghezza?**

FORTRAN : massimo 31 caratteri

C : caratteri illimitati

- ▷ **ci sono caratteri speciali da utilizzare?**

in PHP devono iniziare con il simbolo \$

4.1 Legame

Il concetto di **binding** (legame) viene ereditato dalla matematica.

$$\left(\sum_{i=1}^n \left(\sum_{j=1}^m a_{ij} \dots b_{jk} \right) \right)$$

(a) occorrenza di legame

$$\left(\sum_{i=1}^n \left(\sum_{j=1}^m a_{ij} \dots b_{jk} \right) \right)$$

(b) occorrenza di applicazione

$$\left(\sum_{i=1}^n \left(\sum_{j=1}^m a_{ij} \dots b_{(k)} \right) \right)$$

(c) occorrenza libera

Definizione: occorrenza di legame

L'**occorrenza di legame** (o **definizione**) crea il legame tra nome e il suo significato.

Definizione: occorrenza di applicazione

L'**occorrenza di applicazione** usa il nome per accedere al suo significato.

Definizione: occorrenza libera

L'**occorrenza libera** utilizza un nome a cui non è stato associato nessun significato.

Dal punto di vista del costrutto le occorrenze libere e quelle applicate sono identiche, la differenza consiste nell'essere nel raggio di azione (scope) di una occorrenza di definizione.

Definizione: scope di un binding

Lo **scope** definisce lo spazio in cui si può usare un nome per rappresentare il significato associato a un binding.

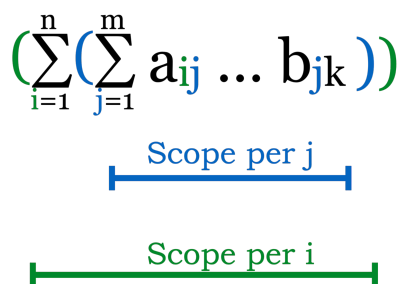


Figura 4.2: scope di un binding

Nei linguaggi di programmazione i concetti di binding e scope prendono una connotazione propria:

- **definizione** (occorrenza di legame): un identificatore è in posizione di definizione quando si definisce il suo significato (si crea il legame)
- **uso** (occorrenza di applicazione): un identificatore è in fase d'uso quando si riferisce al significato associato da una specifica definizione

- **libera** (occorrenza libera): un indentificatore è in posizione libera se il suo uso è nel raggio di azione di nessuna definizione

4.1.1 Tipi di binding

Distingue i binding rispetto alla durata del legame e alla dinamicità del legame.

- ◊ $\left. \begin{array}{l} \text{nome} \leftrightarrow \text{valore} \\ \text{nome} \leftrightarrow \text{locazione} \end{array} \right\}$ legami in cui l'oggetto denotato non cambia (binding statico)
- ◊ $\text{locazione} \leftrightarrow \text{valore} \rightarrow$ legame in cui l'oggetto denotato può cambiare (binding dinamico)

4.1.2 Tempo di binding

Distingue i binding rispetto al momento in cui il binding viene creato.

- ◊ **tempo compilazione** (early binding): FORTRAN, C, COBOL
 - esecuzione veloce
 - non flessibile
 - uso dello stack (allocazione statica)
- ◊ **tempo di esecuzione** (late binding): SNOBOL4, LISP, APL
 - esecuzione lenta
 - flessibile
 - uso dello heap

4.2 Ambiente

I legami vengono inseriti nell'ambiente (insieme di binding).

Le **dichiarazioni** sono il meccanismo (implicito o esplicito) che permette di modificare gli ambienti.

Nelle espressioni abbiamo solo identificatori liberi, ma se vogliamo dare loro significati dobbiamo rendere tali occorrenze applicate inserendole nello scope di binding.

Ci sono due tipi di termini:

- ▷ **termine ground**: termine che non contiene identificatore
- ▷ **termine chiuso**: termine in cui non ci sono identificatori liberi

Entrambi i tipi di termini hanno significato senza dover ricorrere a un contesto esterno (ambiente).

4.3 Sintassi

Riprendiamo la seguente notazione:

- ◊ $\mathcal{E}$: insieme delle espressioni
- ◊ $\mathcal{DVal} = \mathcal{N} \cup \mathcal{B}$: insieme dei valori numerici e booleani (valori denotabili)

Definiamo formalmente l'ambiente per le espressioni:

Definizione: ambiente

L'**ambiente** è un elemento dello spazio delle funzioni:

$$\text{Env} = \bigcup_{V \subseteq_f \text{Id}} \text{Env}_V$$

dove:

- $\text{Env}_V : V \rightarrow \text{Val}$
- $\cup\{\perp\}$ ha una metavariable ρ

A questo punto possiamo completare la sintassi delle espressioni:

Sintassi delle espressioni

```

1  <Exp>
2  E -> A | B
3  A -> I | n | A op A
4  B -> I | true | false | not B | B or B | A = A

```

dove:

- ▷ I: identificatore

4.3.1 Identificatore libero

Data un'espressione o una frase scritta in un certo linguaggio, gli identificatori liberi sono quelli che non hanno nessuna dichiarazione che li coinvolge.

Questo significa che la frase in questione è necessariamente parte di una frase più grande dove gli identificatori liberi sono legati e trovano significato.

Definizione: identificatore libero

La funzione:

$$\mathbf{FI} : \text{Exp} \rightarrow \text{Id}$$

che ad ogni espressione associa l'insieme degli identificatori liberi in essa contenuti, è definita da:

- ▷ $\text{FI} = \emptyset$
- ▷ $\text{FI}(\text{id}) = \{\text{id}\}$
- ▷ $\text{FI}(e_0 \text{ op } e_1) = \text{FI}(e_0) \cup \text{FI}(e_1) = \text{FI}(e_0 \text{ or } e_1) = \text{FI}(e_0 = e_1)$
- ▷ $\text{FI}(\text{not } e_0) = \text{FI}(e)$

Esempio: identificatore libero di un'espressione

Consideriamo l'espressione:

$$\mathcal{E} = x + 5y + 3$$

L'identificatore libero corrisponde a:

$$\begin{aligned} & \text{FI}(x + 5y + 3) \\ &= \text{FI}(x) \cup \text{FI}(5y) \cup \text{FI}(3) \\ &= \{x\} \cup \text{FI}(5y) \cup \text{FI}(3) \\ &= \{x\} \cup \{y\} = \{x, y\} \end{aligned}$$

Così come possiamo definire gli indentificatori liberi, possiamo definire anche quelli definiti **DI** (sempre per induzione sulla struttura sintattica delle espressioni).

La definizione è molto semplice in quanto nelle espressioni non abbiamo costrutti che creano legami (occorrenze), ma solo costrutti che utilizzano occorrenze.

Quindi, l'insieme delle occorrenze legate/definite è vuoto ($\text{DI}(e) = \emptyset$).

4.4 Semantica

4.4.1 Regole di transizione

Definiamo la semantica per le espressioni da valutare in un ambiente ρ :

$$\rho \vdash e \rightarrow e'$$

Ciò significa che l'espressione e viene valutata in e' nell'ambiente ρ .

Le regole di transizione per le espressioni con indentificatori diventano:

▷ **regola 1** (assioma):

Regola $\mathcal{E}_1$ (assioma)

$$\mathcal{E}_1 : \rho \vdash m \text{ op } n \rightarrow p \quad \Leftrightarrow \quad m \text{ op } n = p$$

▷ **regola 2** (assioma):

Regola $\mathcal{E}_2$ (assioma)

$$\mathcal{E}_2 : \rho \vdash I \rightarrow_e n \quad \Leftrightarrow \quad \rho(I) = n$$

▷ **regola 3**:

Regola $\mathcal{E}_3$

$$\mathcal{E}_3 : \frac{\rho \vdash e \rightarrow_e e'}{\rho \vdash e \text{ op } e_0 \rightarrow_e e' \text{ op } e_0}$$

▷ **regola 4**:

Regola $\mathcal{E}_4$

$$\mathcal{E}_4 : \frac{\rho \vdash e \rightarrow_e e'}{\rho \vdash m \text{ op } e \rightarrow_e m \text{ op } e'}$$

▷ **regola 5:**

Regola $\mathcal{E}_5$

$$\mathcal{E}_5 : \frac{\rho \vdash e \rightarrow_e e'}{\rho \vdash \text{not } e \rightarrow_e \text{not } e'}$$

▷ **regola 6:**

Regola $\mathcal{E}_6$

$$\mathcal{E}_6 : \rho \vdash \text{not } t \rightarrow t' = \text{not } t$$

▷ **regola 7:** valuta un identificatore x nell'ambiente ρ

Regola $\mathcal{E}_7$

$$\mathcal{E}_7 : \rho \vdash x \rightarrow k \Leftrightarrow \rho(x) = k$$

4.4.2 Tipo di dati

Introduciamo una nuova informazione al nostro linguaggio: i **tipi**.

Definizione: Tipo

Un **tipo** determina l'insieme di valori omogenei (condividono una certa proprietà strutturale) dotato di un insieme di operazioni definite per i valori di quel tipo.

Esempio: tipi validi

- ◇ Integer
- ◇ String
- ◇ Int $\rightarrow$ bool
- ◇ (Int $\rightarrow$ Int) $\rightarrow$ bool

Esempio: tipi non validi

- ◇ {3, true, x $\rightarrow$ x}
- ◇ Even integers
- ◇ {f: Int $\rightarrow$ Int | x > 3
⇒ f(x) > x * (x + 1)}

I tipi sono utili a vari livelli per diversi motivi:

- ▷ **livello di progetto:** organizzano l'informazione (tipi diversi per concetti diversi costituiscono "commenti")

- ▷ **livello di programma:** identificano e prevengono errori
- ▷ **livello di implementazione:** permettono un'ottimizzazione della memoria

4.4.2.1 Legami di tipo

Il tipo diventa un oggetto denotabile e, quindi, associabile agli identificatori.

In generale il tipo non viene associato all'identificatore per essere riferito come oggetto, ma per descrivere proprietà che permettono di descrivere vincoli semantici, non descrivibili mediante la grammatica del linguaggio (vincoli contestuali).

Per capire meglio il concetto di tipo e di legame di tipo è importante porsi tre domande:

1. **come si specifica il tipo?**
2. **quando avviene il binding?** (tempo di esecuzione o compilazione)
3. **come avviene creato il legame?** (espliciti/dichiarazioni o impliciti/assegnamenti)

4.4.3 Semantica statica

La semantica statica è lo strumento formale che permette di creare legami di tipo e permette di verificare che i vincoli contestuali siano verificati.

4.4.3.1 Ambiente statico

L'ambiente statico colleziona i legami di tipo.

Definizione: ambiente statico

L'**ambiente statico** è un elemento dello spazio delle funzioni $\mathcal{TEnv}$ definita da:

$$\mathcal{TEnv} = \bigcup_{V \subseteq_f \text{Id}} \mathcal{TEnv}_V$$

dove:

- ▷ $\mathcal{TEnv}_V : V \rightarrow \mathcal{DTyp}$: ha metavariable Δ
- ▷ $\mathcal{DTyp}$: insieme dei tipi denotabili

4.4.3.2 Semantica statica delle espressioni

I tipi denotabili nel nostro linguaggio sono i valori interi e booleani:

$$\tau \in \mathcal{DTyp} = \{\text{int}, \text{bool}\}$$

Le regole della semantica statica hanno la seguente forma:

Assioma

$$\mathcal{E}_{S_1} : \vdash e : \tau$$

⇒ **assioma**: senza necessità di specificare l'ambiente statico (vali-

do per ogni ambiente statico) possiamo determinare il tipo τ dell'espressione e

Regola

$$\mathcal{E}_{S_1} : \Delta \vdash_V e : \tau$$

⇒ **regola**: nell'ambiente statico contestuale Δ possiamo deter-

minare il tipo τ dell'espressione e

Vediamo, allora, come sono fatte le regole della semantica statica nel nostro linguaggio:

- ▷ **regola 1 e 2** (assioma): in ogni tipo di ambiente un numero ha tipo intero (`int`) e una costante booleane ha tipo booleano (`bool`)

Regola $\mathcal{E}_{S_1}$ (assioma)

$$\mathcal{E}_{S_1} : \vdash n : \text{int}$$

Regola $\mathcal{E}_{S_2}$ (assioma)

$$\mathcal{E}_{S_1} : \vdash t : \text{bool}$$

- ▷ **regola 3**: determina che l'identificatore ha come tipo proprio quello che l'ambiente statico Δ del contesto gli associa

Regola $\mathcal{E}_{S_3}$

$$\mathcal{E}_{S_3} : \Delta \vdash_V I : \tau \Leftrightarrow \Delta(I) = \tau, I \in V$$

- ▷ **regola 4**:

Regola $\mathcal{E}_{S_4}$

$$\mathcal{E}_{S_4} : \frac{\Delta \vdash_V e_1 : \text{int} \quad \Delta \vdash_V e_2 : \text{int}}{\Delta \vdash_V e_1 \text{ op } e_2 : \text{int}} \quad \text{op} \in \{+, -, *\}$$

- ▷ **regola 5**:

Regola $\mathcal{E}_{S_5}$

$$\mathcal{E}_{S_5} : \frac{\Delta \vdash_V e_1 : \text{bool} \quad \Delta \vdash_V e_2 : \text{bool}}{\Delta \vdash_V e_1 \text{ or } e_2 : \text{bool}} \quad \text{op} \in \{+, -, *\}$$

- ▷ **regola 6**:

Regola $\mathcal{E}_{S_6}$

$$\mathcal{E}_{S_6} : \frac{\Delta \vdash_V e_0 : \text{bool}}{\Delta \vdash_V \text{not } e_0 : \text{bool}}$$

▷ regola 7:

Regola $\mathcal{E}_{S_6'}$

$$\mathcal{E}_{S_6'} : \frac{\Delta \vdash_V e_1 : \tau \quad \Delta \vdash_V e_2 : \tau}{\Delta \vdash_V e_1 = e_2 : \text{bool}} \quad \tau = \text{tipi uguali}$$

4.4.3.3 Compatibilità di ambienti

Ora abbiamo due concetti di ambiente diverso che parlano degli stessi identificatori. È chiaro che possiamo aspettarci che i due ambienti definiti sullo stesso programma parlino in modo coerente degli stessi identificatori.

Definiamo, quindi, il concetto di **compatibilità tra ambienti** (statico e dinamico).

Definizione: compatibilità tra ambienti

Sia $\rho : V$ un ambiente dinamico e $\Delta : V$ un ambiente statico con $V \subseteq_f \text{Id}$.
Gli ambienti ρ e Δ sono compatibili ($\rho : \Delta$) se e soltanto se:

$$\forall I \in V. (\Delta(I) = \tau \wedge \rho(I) \in \tau)$$

Un ambiente statico e uno dinamico sono compatibili se parlano in modo coerente degli stessi identificatori.

Dichiarazioni

Le **dichiarazioni** sono la categoria sintattica che denota la creazione/modifica di ambienti (insieme di legami).

Inoltre, sono lo strumento sintattico per manipolare e creare di ambienti (insieme di legami tra nomi e oggetti denotabili).

Le dichiarazioni devono essere elaborate per ottenere trasformazioni (reversibili) degli ambienti.

5.1 Sintassi

Sintassi delle dichiarazioni

```

1  <Dec>
2  D -> nil | \rho | const x : \tau = e | D ; D | D in D

```

dove:

- ▷ **nil**: dichiarazione vuota (non crea nessun ambiente e non genera trasformazioni)
- ▷ ρ : ambiente dinamico $\equiv$ dichiarazione completamente dichiarata
- ▷ **const** $x : \tau = e$: dichiarazione di un nome associato a un valore e
- ▷ **D ; D**: composizione sequenziale.
Ogni dichiarazione successiva o aggiunge nuovi legami o sovrascrive legami esistenti.
L'ambiente risultante è così visibile al codice che viene successivamente



Esempio: composizione sequenziale

Consideriamo il seguente codice:

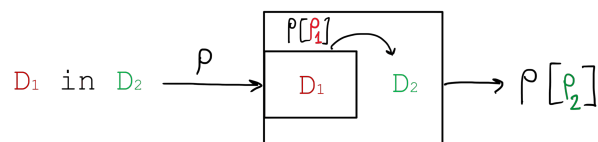
```
const x: int = 3;
const x: int = 5;      const y: int = 6 * x;
const z: int = z + y;
```

Osserviamo i valori che assumono le variabili di questo codice:

1. a riga 1 il valore assegnato a x è $\rho = [x \mapsto 3]$
2. a riga 2 il valore assegnato a x è $\rho_1 = [x \mapsto 5]$ e viene usata per elaborare y che assume valore $\rho_2 = [y \mapsto 6 * 5 = 30]$
3. a riga 3 viene usati il valore di x dichiarato a riga 2 e il valore di y dichiarato a riga 2 ($\rho_2 = [x \mapsto 5, y \mapsto 30]$) per elaborare z che assume il valore ($\rho_3 = [x \mapsto 5, y \mapsto 30, z \mapsto 5 + 30 = 35]$)

▷ **D in D**: composizione privata.

I legami creati o sovrascritti da D_1 sono ad uso esclusivo dell'elaborazione di D_2 e non visibili al codice successivo

**Esempio: composizione privata**

Consideriamo il seguente codice:

```
const x: int = 3;
const x: int = 5;      const y: int = 6 * x;
const z: int = z + y;
```

Osserviamo i valori che assumono le variabili di questo codice:

1. a riga 1 il valore assegnato a x è $\rho = [x \mapsto 3]$
2. a riga 2 il valore assegnato a x è $\rho_1 = [x \mapsto 5]$ e viene usata per elaborare y che assume valore $\rho_2 = [y \mapsto 6 * 5 = 30]$
3. a riga 3 viene usati il valore di x dichiarato a riga 1 e il valore di y dichiarato a riga 2 ($\rho_2 = [x \mapsto 5, y \mapsto 30]$) per elaborare z che assume il valore ($\rho_3 = [x \mapsto 5, y \mapsto 30, z \mapsto 3 + 30 = 33]$)

`const x:int=3;`
`const x:int=5 in const y:int=6*x;`
`var z:int=x+y`

`const x:int=3;`
`const x:int=5 in const y:int=6*x;`
`var z:int=x+y`

5.1.1 Modifica dell'ambiente

Consideriamo gli ambienti (dinamici):

$$\beta, \beta' \in \text{Env} \quad \beta : V, \beta' : V' \quad V, V' \subseteq \text{Id}$$

La modifica di β da parte di β' si scrive:

$$\beta[\beta']$$

Il nuovo ambiente $\beta'' (= \beta[\beta']) \in \text{Env}$ sarà:

$$\beta''(I) = \begin{cases} \beta'(I) & \text{se } I \in V \\ \beta(I) & \text{se } I \in V, V' \end{cases}$$

Ciò significa che parlerà sia degli identificatori in I che in quelli in I' :

$$\beta[\beta'] = \hat{\beta} \hat{\beta} : I \cup I'$$

Esempio: modifica dell'ambiente

Consideriamo i seguenti ambienti:

$$\beta = [z \mapsto 5, y \mapsto \text{false}]$$

$$\beta' = [x \mapsto 3, y \mapsto \text{true}]$$

La modifica di β da parte di β' è il seguente:

$$\beta[\beta'] = [x \mapsto 3, y \mapsto \text{true}]$$

Il valore di y viene **sovrascritto** perchè è presente in entrambi gli ambienti.

5.1.2 Identificatori definiti

Gli identificatori definiti (da una dichiarazione) sono gli identificatori che trovano il significato associato dentro la dichiarazione.

Definizione: identificatore definito (formale)

Definiamo la funzione DI che associa ad ogni dichiarazione l'insieme degli identificatori:

$$\mathbf{DI} : \mathbf{Dec} \rightarrow \mathcal{P}(\mathbf{Id})$$

- ▷ $\mathbf{DI}(\mathbf{nil}) = \emptyset$
- ▷ $\mathbf{DI}(\rho) = V \quad \rho : V$ (attribuisce significato a tutti gli elementi del suo dominio)
- ▷ $\mathbf{DI}(\mathbf{const} \quad x : \tau = e) = \{x\}$ (attribuisce significato all'identificatore che sta dichiarando)
- ▷ $\mathbf{DI}(d_0; d_1) = \mathbf{DI}(d_0) \cup \mathbf{DI}(d_1)$ (attribuisce significato a tutti gli identificatori a cui viene dato significato in d_0 e d_1)
- ▷ $\mathbf{DI}(d_0 \text{ in } d_1) = \mathbf{DI}(d_2)$ (ciò che viene definito nella prima dichiarazione d_1 rimane privato e, quindi, non risulta legato all'esterno della composizione. Fuori dalla composizione è legato solo ciò che viene definito nella seconda composizione d_2)

5.1.3 Identificatori liberi

GLi identificatori liberi sono identificatori il cui significato va cercato esternamente (nel contesto) alla dichiarazione.

Definizione: identificatore libero (formale)

Definiamo la funzione FI che applicata ad una dichiarazione restituisce l'insieme degli identificatori liberi nella dichiarazione:

$$\mathbf{FI} : \mathbf{dec} \rightarrow \mathcal{P}(\mathbf{Id})$$

- ▷ $\mathbf{FI}(\mathbf{nil}) = \emptyset$
- ▷ $\mathbf{FI}(\rho) = \emptyset$
- ▷ $\mathbf{FI}(\mathbf{const} \quad x : \tau = e) = \mathbf{FI}(e)$
- ▷ $\mathbf{FI}(d_0; d_1) = \mathbf{FI}(d_1 \text{ in } d_2) = \mathbf{FI}(d_1) \cup (\underbrace{\mathbf{FI}(d_2)}_{\text{togliere}} \setminus \mathbf{DI}(d_1))$

5.2 Semantica**5.2.1 Semantica statica****Definizione: semantica statica**

La **semantica statica** verifica se una dichiarazione è ben formata (rispetta vincoli contestuali).

La semantica statica per per dichiarazioni associa un ambiente statico Δ ad ogni dichiarazione ben formata.

Le regole hanno la seguente forma:

$$\mathcal{D}_{S_i} : \Delta \vdash_v d : \Delta$$

Vediamo, quindi, le regole:

- ▷ **regola 1** (assioma per gli ambienti statici):

Assioma $\mathcal{D}_{S_1}$

$$\mathcal{D}_{S_1} : \vdash \text{null} : \emptyset$$

- ▷ **regola 2** (assioma per gli ambienti dinamici):

▷

Assioma $\mathcal{D}_{S_2}$

$$\mathcal{D}_{S_2} : \vdash \rho : \Delta \quad \Leftrightarrow \quad \rho \vdash \Delta$$

- ▷ **regola 3:**

Regola $\mathcal{D}_{S_3}$

$$\mathcal{D}_{S_3} : \frac{\Delta \vdash_v e : \tau}{\Delta \vdash_v \text{const } x : \tau = e : [x \leftarrow \tau]}$$

- ▷ **regola 4** (dichiarazione composta):

Regola $\mathcal{D}_{S_4}$

$$\mathcal{D}_{S_4} : \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta_1[\Delta_2] \vdash d_2 : \Delta_2}{\Delta \vdash d_1; d_2 : \Delta_1[\Delta_2]}$$

- ▷ **regola 5** (dichiarazione composta):

Regola $\mathcal{D}_{S_5}$

$$\mathcal{D}_{S_5} : \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta_1[\Delta_2] \vdash d_2 : \Delta_2}{\Delta \vdash d_1 \text{ in } d_2 : \Delta_2}$$

5.2.2 Semantica dinamica

Definizione: semantica dinamica

La **semantica dinamica** consiste nell'elaborazione che parte dall'assunzione che la dichiarazione è ben formata.

Le regole di transizione hanno la forma:

$$\mathcal{D}_i : \rho \vdash_{\Delta} d \rightarrow_d d'$$

dove:

- ▷ ρ : ambiente nel quale elaboriamo la dichiarazione (compatibile con Δ)
- ▷ d : dichiarazione che viene elaborata in d' nell'ambiente ρ

Vediamo, quindi, le regole:

- ▷ **regola 1** (assioma): ρ è già terminale (non c'è nulla da elaborare)

Assioma $\mathcal{D}_{S_1}$

$$\mathcal{D}_1 : \vdash \text{nil} \rightarrow_d \emptyset$$

- ▷ **regola 2** (assioma):

Assioma $\mathcal{D}_{S_2}$

$$\mathcal{D}_2 : \rho \vdash \text{const } x : \tau = k \rightarrow_d [x \mapsto k]$$

- ▷ **regola 3**:

Regola $\mathcal{D}_{S_3}$

$$\mathcal{D}_3 : \frac{\rho \vdash \Delta e \rightarrow_e e'}{\rho \vdash_{\Delta} \text{const } x : \tau = e \rightarrow_d \text{const } x : \tau = e'}$$

- ▷ **regola 4**:

Regola $\mathcal{D}_{S_4}$

$$\mathcal{D}_4 : \frac{\rho \vdash \Delta d \rightarrow_e d'}{\rho \vdash_{\Delta} d; d_1 \rightarrow_d d'; d_1}$$

- ▷ **regola 5**:

Regola $\mathcal{D}_{S_5}$

$$\mathcal{D}_5 : \frac{\rho[\rho_0] \vdash \Delta d_1 \rightarrow_e d'_1}{\rho \vdash_{\Delta} \rho_0; d_1 \rightarrow_d \rho_0; d'_1}$$

- ▷ **regola 6**:

Regola $\mathcal{D}_{S_6}$

$$\mathcal{D}_6 : \rho \vdash_{\Delta} \rho_0; \rho_1 \rightarrow_d \rho_0[\rho_1]$$

▷ **regola 7:**

Regola $\mathcal{D}_{S_7}$

$$\mathcal{D}_7 : \frac{\rho \vdash \Delta d \rightarrow_e d'}{\rho \vdash_{\Delta} d \text{ in } d_1 \rightarrow_d d \text{ in } d_1}$$

▷ **regola 8:**

Regola $\mathcal{D}_{S_8}$

$$\mathcal{D}_8 : \frac{\rho[\rho_0] \vdash \Delta d_1 \rightarrow_e d'_1}{\rho \vdash_{\Delta} \rho_0 \text{ in } d_1 \rightarrow_d \rho_0 \text{ in } d'_1}$$

▷ **regola 9:**

Regola $\mathcal{D}_{S_9}$

$$\mathcal{D}_9 : \rho \vdash_{\Delta} \rho_0 \text{ in } \rho_1 \rightarrow_d \rho_0[\rho_1]$$

Definizione: elaborazione delle dichiarazioni

La **valutazione** è una funzione

$$\text{Elab} : \mathcal{D} \rightarrow \text{Env}$$

che descrive il comportamento dinamico delle dichiarazioni restituendo l'ambiente in cui esse sono elaborate:

$$\text{Elab}(d) = \rho \quad \Leftrightarrow \quad \rightarrow * \rho$$

La definizione di valutazione di una dichiarazione permette, anche, di definire il concetto di equivalenza tra dichiarazioni.

Definizione: equivalenza di dichiarazioni

L'**equivalenza di dichiarazioni** è una relazione

$$\equiv \subseteq \mathcal{D} \times \mathcal{D}$$

definita come segue:

$$d_0 \equiv d_1 \quad \Leftrightarrow \quad \text{Elab}(d_0) = \text{Elab}(d_1)$$

Variabili

I **comandi** sono la categoria sintattica che permettono di rappresentare trasformazioni irreversibili dello stato.

L'astrazione dello stato della macchina è rappresentata dalla memoria.

Definizione: memoria

La **memoria** è l'insieme delle associazioni tra locazioni e valori (es. array illimitato: indice = locazione, valore = contenuto della locazione).

La trasformazione dello stato, avviene tramite un meccanismo definito **assegnamento**.

Definizione: assegnamento

L'**assegnamento** è un comando che permette di modificare il contenuto della memoria.

Ne deriva che non è pensabile che si possa utilizzare direttamente la locazione di memoria. Le locazioni vengono riferite attraverso identificatori chiamate variabili.

Definizione: variabili

Le **variabili** sono identificatori associati a locazioni, che a loro volta, nella memoria, sono associate a valori che possono cambiare durante l'esecuzione.

Per il momento, quindi, supponiamo che le variabili siano nomi astratti per le celle di e supponiamo di associare direttamente il valore (modificabile) a una variabile.

Vediamo alcuni aspetti dell'uso delle variabili che rendono questo modello poco adatto:

- ▷ quando ci riferiamo a un valore modificabile mediante una variabile, lo possiamo fare sia per accedere al suo valore in lettura, sia per modificare il suo valore
- ▷ per riferirsi a oggetti da modificare potremmo utilizzare funzioni della variabile e non il semplice nome (es. array $a[x+1]$)
- ▷ gli identificatori possono essere ridefiniti causando modifiche reversibili all'ambiente. Ciò significa che per un certo intervallo di tempo, una variabile potrebbe avere in altro significato, mentre il valore che aveva prima della ridefinizione non deve andare perso perchè potrebbe ritornare valido (es. uscita da una procedura)

Introduciamo, quindi, la locazione come astrazione della memoria fisica e la variabile come astrazione della locazione.

Significa che introduciamo qualcosa di intermedio tra il nome della variabile e il nome riferito, che modelli la memoria nella sua visione ideale (illimitata).

In questo modo è possibile che una variabile possa avere indirizzi differenti in momenti diversi dall'esecuzione, oppure che una variabile possa avere differenti indentificatori in punti diversi del programma.

Si distinguono l'associazione tra nome e locazione che non cambia in modo reversibile durante l'esecuzione e l'associazione tra locazione e valore che cambia in modo irreversibile durante l'esecuzione.

Definizione: locazione

La **locazione** è il contenitore di valori che possono essere inutilizzati, indefiniti o definiti.

Definizione: memoria

La **memoria** è la collocazione delle associazioni tra locazioni e valori contenuti, che possono esser costantaneamente cambiati.

Questo significa che la locazione viene creata da una dichiarazione.

Inoltre, la locazione ha un tempo di vita determinato dallo scope della dichiarazione.

Definizione: identificatore costante

un **identificatore costante** è un identificatore che può assumere un valore predefinito e non è ulteriormente modificabile.

Definizione: identificatore variabile

Un **identificatore variabile** è un identificatore il cui valore può essere aggiornato.

Comandi

I **comandi** sono la categoria sintattica il cui ruolo principale consiste nella trasformazione (irreversibile) della memoria.

I comandi devono essere eseguiti per attuare le richieste di trasformazione della memoria che rappresentano.

7.1 Sintassi

Abbiamo introdotto un nuovo tipo di identificatore (variabile), che deve essere legata ad una locazione.

Questo significa che dobbiamo aggiungere una dichiarazione alla grammatica delle dichiarazioni che permetta di definire questo nuovo oggetto sintattico.

Sintassi delle dichiarazioni

```

1   <Dec>
2   D -> nil | \rho | const x : \tau = e | D ; D | D in D |
3       var x : \tau = e

```

La nuova dichiarazione definisce un identificatore x come una variabile di tipo τ e inizializzata la variabile al valore denotato all'espressione e .

7.2 Semantica

7.2.1 Semantica statica

Abbiamo appena modificato la grammatica delle dichiarazioni, quindi, per prima cosa dobbiamo aggiungere la regola di semantica statica per il nuovo costrutti di dichiarazione di variabile.

Prima, però, bisogna aggiungere un nuovo tipo che ci permetta di distinguere tra identificatori variabili e costanti.

Definiamo, quindi, il tipo τ_{loc} utilizzato per gli identificatori variabili di tipo τ :

$$\tau \in \mathcal{D}Typ = \{\text{int}, \text{bool}, \text{intloc}, \text{boolloc}\}$$

Le regole di semantica sono le stesse delle dichiarazioni, senza nessuna modifica. Dobbiamo, però, aggiungere la regola per la nuova dichiarazione.

Regola $\mathcal{D}_{S_6}$

$$\mathcal{D}_{S_6} : \frac{\Delta \vdash e : \tau}{\Delta \vdash \text{var } x : \tau = e : [x \leftarrow \tau \text{loc}]}$$

Ricordiamo l'assioma per gli identificatori:

$$\Delta \vdash x : \tau \Leftrightarrow \Delta(x) = \tau$$

dove τ è il tipo associato a x .

Ora che abbiamo aggiunto le locazioni nei tipi, bisogna gestire anche i tipi delle locazioni e la nuova regola per gli identificatori diventa:

$$\Delta \vdash x : \tau \Leftrightarrow \Delta(x) \in \{\tau, \tau \text{loc}\}$$

7.2.2 Memoria

Le locazioni sono modellate come un insieme finito, ma illimitato di elementi che rappresentano celle di memoria.

- ▷ Loc : insieme delle locazioni ($l \in \text{Loc}$)
- ▷ $\text{New}(L) \in \text{Loc}$
 L : restituisce una nuova locazione, che non appartiene a L ($L \subseteq \text{Loc}$)

Definizione: memoria

Sia $\mathcal{M}\text{Val}$ l'insieme dei valori memorizzabili (valori associabili a una locazione):

$$\mathcal{M}\text{Val} = \mathcal{N} \cup \mathcal{B}$$

Una memoria è un elemento dello spazio di funzioni Mem definito da:

$$\text{Mem} = \bigcup_{L \subseteq_f \text{Id}} \text{Mem}_L$$

dove:

- ▷ $\text{Mem}_L : L \rightarrow \mathcal{M}\text{Val}$: ha metavariable σ

Cambia anche il concetto di ambiente visto che ora si possono associare agli identificatori le locazioni.

Definizione: ambiente

Un **ambiente** è una funzione che associa ad ogni identificatore un valore o locazione:

$$\text{Env}_V : V \rightarrow \mathcal{N} \cup \mathcal{B} \cup \mathcal{L} \quad V \subseteq_f \text{Id}$$

$$\text{Env} = \bigcup_{V \subseteq_f \text{Id}} \text{insieme di tutte le funzioni ambiente}$$

7.2.3 Aggiornamento della memoria

Si considerino due memorie $\sigma, \sigma' \in \text{Mem}$ dove $\sigma : L, \sigma' : L'$ (σ è definito sulle locazioni in L e σ' è definito sulle locazioni in L').

L'aggiornamento della memoria σ è la memoria $\sigma'' \in \text{Mem}$ ($\sigma[\sigma']$) definito come:

$$\sigma''(l) = \begin{cases} \sigma'(l) & \text{se } l \in L' \\ \sigma(l) & \text{altrimenti} \end{cases}$$

7.2.4 Semantica dinamica

Introdurre le variabili significa poter scrivere espressioni e, quindi, dichiarazioni che contengono identificatori variabili.

Questo implica che per valutare queste espressioni ed elaborare queste dichiarazioni abbiamo bisogno di interagire nel sistema di transizione anche la memoria da usare per valutare gli identificatori variabili.

Le regole diventano:

$$\mathcal{E}_i : \rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle$$

L'unica regola che cambia è quella di valutazione degli identificatori, che deve considerare il caso in cui l'identificatore sia una variabile.

Regola $\mathcal{E}_2$

$$\mathcal{E}_{S_2} : \rho \vdash_{\Delta} \langle I, \sigma \rangle \rightarrow_e \langle n, \sigma \rangle \quad \Leftrightarrow \quad \rho(I) \text{ o } (\rho(I) = \text{le}\sigma(I) = n)$$

Per tutte le altre, semplicemente aggiungiamo la memoria nella regola, ma osserviamo che tale memoria non può essere trasformata dalla valutazione, che accade in memoria sempre e solo in lettura.

Per ciò che riguarda le dichiarazioni.

Definizione: sistema di transizione

$$\tau^V = \mathcal{D}^V \times \text{Mem}$$

$$T^V = \text{Env} \times \text{Mem}$$

Le regole diventano:

$$\mathcal{D}_i : \rho \vdash_{\Delta} \langle \mathbf{d}, \sigma \rangle \rightarrow_e \langle \mathbf{d}', \sigma' \rangle$$

In questo caso la transizione la memoria può essere diversa, questo perchè le dichiarazioni, inizializzando le variabili modificando la memoria.

Le regole sono:

▷ **regola 1:**

Regola $\mathcal{E}_1$

$$\mathcal{E}_1 : \rho \vdash_{\Delta} \langle m \text{ op } n, \sigma \rangle \rightarrow_e \langle k, \sigma \rangle \quad \Leftrightarrow \quad m \text{ op } n = k, n \in \mathcal{N}, k \in \mathcal{N} \cup \mathcal{B}$$

▷ **regola 2:**

Regola $\mathcal{E}_2$

$$\mathcal{E}_2 : \rho \vdash_{\Delta} \langle I, \sigma \rangle \rightarrow_e \langle n, \sigma \rangle \quad \Leftrightarrow \quad \rho(I) \text{ o } (\rho(I) = l \text{ e } \sigma(I) = n)$$

▷ **regola 3:**

Regola $\mathcal{E}_3$

$$\mathcal{E}_3 : \frac{\rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle}{\rho \vdash_{\Delta} \langle e \text{ op } e_0, \sigma \rangle \rightarrow_e \langle e' \text{ op } e_0, \sigma \rangle}$$

▷ **regola 4:**

Regola $\mathcal{E}_4$

$$\mathcal{E}_4 : \frac{\rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle}{\rho \vdash_{\Delta} \langle m \text{ op } e, \sigma \rangle \rightarrow_e \langle m \text{ op } e', \sigma \rangle}$$

▷ **regola 5:**

Regola $\mathcal{E}_5$

$$\mathcal{E}_5 : \rho \vdash_{\Delta} \langle t_1 \text{ bop } t_2, \sigma \rangle \rightarrow_e \langle t, \sigma \rangle \quad \Leftrightarrow \quad t_1 \text{ op } t_2, t_1, t_2, t \in \mathcal{B}$$

▷ **regola 3':**

Regola $\mathcal{E}'_3$

$$\mathcal{E}'_3 : \frac{\rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle}{\rho \vdash_{\Delta} \langle e \text{ bop } e_0, \sigma \rangle \rightarrow_e \langle e \text{ bop } e_0, \sigma \rangle}$$

▷ **regola 6:****Regola $\mathcal{E}_6$**

$$\mathcal{E}_6 : \frac{\rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle}{\rho \vdash_{\Delta} \langle m \text{ bop } e_0, \sigma \rangle \rightarrow_e \langle m \text{ bop } e', \sigma \rangle}$$

▷ **regola 7:****Regola $\mathcal{E}_7$**

$$\mathcal{E}_7 : \rho \vdash_{\Delta} \langle \text{not } t_1, \sigma \rangle \rightarrow_e \langle t, \sigma \rangle \quad \Leftrightarrow \quad \text{not } t_1 = t, \quad t, t_1 \in \mathcal{B}$$

▷ **regola 8:****Regola $\mathcal{E}_8$**

$$\mathcal{E}_8 : \frac{\rho \vdash_{\Delta} \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle}{\rho \vdash_{\Delta} \langle \text{not } e, \sigma \rangle \rightarrow_e \langle \text{not } e', \sigma \rangle}$$

7.3 Strutture di controllo

7.3.1 Comandi di selezione

I comandi di selezione forniscono gli strumenti per scegliere tra due o più cammini di esecuzione:

- ▷ **selettori a due vie** (if-then-else)
- ▷ **selettori a più vie** (switch-case)

7.3.1.1 Selettori a due vie

I comandi condizionali presentano la seguente sintassi:

Sintassi comandi condizionali

```
1  if B then C else C
```

Prima, però, vediamo in che posizione si trovano gli identificatori dentro questo costrutto:

$$FI(\text{if } B \text{ then } C \text{ else } C) = FI(B) \cup FI(C) \cup FI(C)$$

$$DI(\text{if } B \text{ then } C \text{ else } C) = DI(C) \cup DI(C)$$

7.3.1.1.1 Semantica statica

7.3.1.1.2 Semantica dinamica

7.3.1.2 Selettori a più vie

Procedure

L'uso delle procedure permette l'**astrazione del controllo**.

Nota

Astrarre significa nascondere dettagli facendo emergere caratteristiche comuni.

L'astrazione chiede di identificare le proposizioni importanti che si vuole descrivere al fine di concentrarsi che si vuole descrivere al fine di concentrarsi sulle questioni rilevanti e ignorare quelle non importanti.

Quando si parla di astrazione del controllo ci sono, in realtà, due categorie di sottoprogrammi con nome che possiamo definire:

- ◇ **procedure** (no output)
- ◇ **funzioni** (restituisce output)

Parte importante del processo di astrazione consiste nel dare al raggruppamento un nome.

Il **nome** permette di riferirsi all'astrazione senza conoscerne i dettagli, una sola le caratteristiche di interesse che ne hanno determinato il raggruppamento.

I **parametri** permettono l'astrazione di tante computazioni simili che cambiano di tante computazioni simili che cambiano solo per alcuni input a cui vengono applicate e che vengono abbreviate con un singolo nome.

Il **corpo** è ciò che viene valutato/eleborato/eseguito ogni volta che il nome viene richiamato.

Una procedura viene definita in tre fasi:

1. **specifica**: descrizione dell'interfaccia della procedura/funzione

```

1      double P(int x) {
2
3      }
4
```

2. **definizione**: descrizione del corpo della procedura/funzione

```

1      {
2          double y;
3          //corpo procedura P

```

```

4      ...
5      (return y);
6  }
7

```

3. **uso**: chiamata della procedura/funzione per eseguire il codice

```

1      int n;
2      double z;
3      z = P(n); //chiamata della procedura
4

```

Consideriamo il seguente codice:

```

1      int foo(int n) {
2          int tmp = a;
3
4          if(tmp == 0)
5              return n;
6          else
7              return n + 1;
8      }

```

Listing 8.1: programma di esempio

In questo caso la variabile `a` è un identificatore non locale, cioè un identificatore libero per cui di deve trovare il significato nel contesto.

Il problema è che nel contesto del chiamante potrebbero esserci più definizioni di `a` e non è chiaro quale sia quella a cui si riferisce la procedura.

Di conseguenza, la regola di visibilità delle dichiarazioni non è più sufficiente per risolvere i riferimenti quando si usano le procedure se la procedura ha un ambiente non locale.

Definizione: Ambiente di una procedura

L'**ambiente di riferimento** di una procedura è l'insieme di tutti i nomi che sono visibili al comando.

L'ambiente di riferimento di una procedura è determinato da varie tipologie di regole:

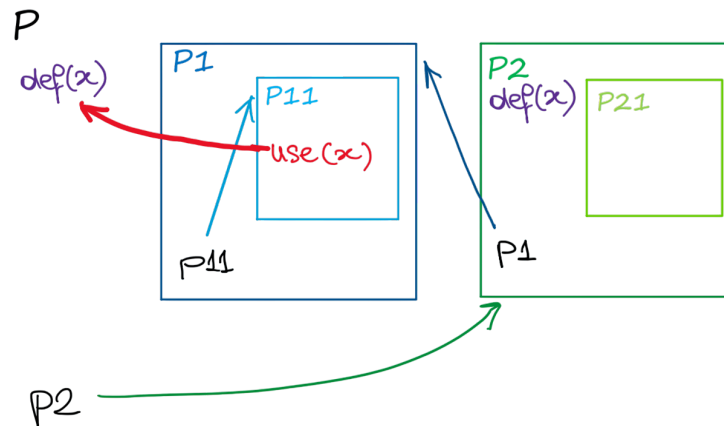
- ◇ **regole di scope** (statico o dinamico)
- ◇ **regole di visibilità**: determinano quali dichiarazioni sono visibili in un certo punto del programma
- ◇ **regole di binding** (shallow o deep): determinano a quale dichiarazione si riferisce un certo identificatore

8.1 Regole di scope

Le regole di scope possono essere distinte in due tipologie:

- ▷ **scope statico**: prevale il contesto di definizione.
L'ambiente di riferimento è quello attivo al momento della definizione della procedura.
- ▷ **scope dinamico**: prevale il contesto di esecuzione.
L'ambiente di riferimento è quello attivo al momento della chiamata della procedura.

8.1.1 Scope statico



Con lo scope statico il riferimento per x è P , ovvero si risale la catena di annidamento dei blocchi (**catena statica**), nota a tempo di compilazione.

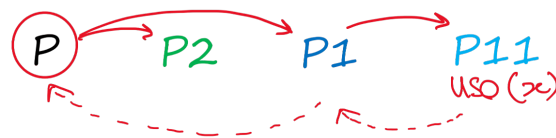


Figura 8.1: catena di definizione

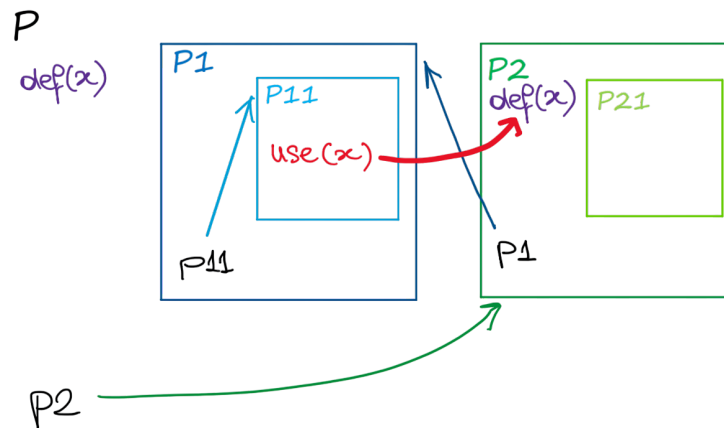
```

1  {
2      int x = 0;
3
4      void pippo(int n) {
5          x = n + 1;
6      }
7
8      pippo(3);
9      write(x); //stampa in ogni caso 4
10
11     {
12         int x = 0;
13         pippo(3);
14         write(x);
15         //stampa 0 perche' pippo(3) va a modifica il primo valore
16         //di x, ma la write si riferisce all'ultima istanza (x = 0)
17     }
18
19     write(x); //stampa in ogni caso 4
20 }

```

Listing 8.2: codice esempio con scope statico

8.1.2 Scope dinamico



Con lo scope dinamico il riferimento per x è $P2$. ovvero si risale la catena di chiamate dei blocchi (**catena dinamica**), nota a tempo di compilazione.



Figura 8.2: catena di definizione

```

1  {
2      int x = 0;
3
4      void pippo(int n) {
5          x = n + 1;
6      }
7
8      pippo(3);
9      write(x); //stampa in ogni caso 4
10
11     {
12         int x = 0;
13         pippo(3);
14         write(x);
15         //stampa 4 perche' pippo(3) va a modifica il secondo valore
16         //di x
17     }
18
19     write(x); //stampa in ogni caso 4
20 }

```

Listing 8.3: codice esempio con scope dinamico

8.2 Visibilità delle dichiarazioni

Una dichiarazione può essere visibile:

▷ a partire dalla dichiarazione e fino alla fine del blocco

```
1      {  
2          a := ...  
3          .  
4          .  
5          .  
6          int i := int //visibile da qui in giu'  
7      }  
8
```

▷ all'interno del blocco visibile sempre